

FULL IMPLEMENTATION GUIDE

Cloud Data Platform

Step-by-Step Build Manual

VPC · EKS · Operators · Ingestion · Lakehouse · GitOps

 GitHub Actions CI/CD

 Kubernetes Operators

 MinIO + Iceberg Lakehouse

 Spark + dbt Compute

 Cloudflare Zero Trust

 Prometheus + Grafana

 Terraform + Ansible IaC

 ArgoCD GitOps

Released by: Mohammed Dahiry · Academic Year 2025–2026

Contents

1	Prerequisites & Repository Bootstrap	3
1.1	Required Tools Installation	3
1.2	AWS Credentials & IAM Setup	4
1.2.1	Minimum IAM Permissions for Terraform	5
1.3	Git Repository Structure	6
1.4	Cloudflare R2 – Terraform Remote State Backend	7
2	Phase 1 – VPC, Networking & EKS Cluster	9
2.1	CIDR Design & Subnet Strategy	9
2.2	Terraform VPC Module	9
2.3	Security Groups	11
2.4	EKS Cluster Terraform Module	12
2.5	Environment Root Configuration	16
2.6	Deploying Phase 1 Infrastructure	17
2.6.1	Create gp3 Storage Class	18
2.7	Kubernetes Namespace & RBAC Setup	19
3	Phase 2 – Storage Layer: MinIO, Hive & Databases	22
3.1	cert-manager (TLS Prerequisites)	22
3.2	MinIO Operator Installation	22
3.3	MinIO Tenant (Lakehouse Storage)	23
3.3.1	Create MinIO Credentials Secret	23
3.4	PostgreSQL via CloudNativePG	25
3.5	Hive Metastore Deployment	27
4	Phase 3 – Ingestion: Kafka, NiFi & CDC	30
4.1	Strimzi Kafka Operator	30
4.2	Apache NiFi Deployment	32
4.2.1	Configuring the CDC Flow in NiFi	34
5	Phase 4 – Compute: Spark, dbt & Airflow	36
5.1	Spark Operator	36
5.1.1	Spark RBAC & Service Account	36
5.1.2	Spark Docker Image with Iceberg	37
5.1.3	Bronze Layer Iceberg Writer Job	38
5.2	dbt Project Setup	40

5.3	Apache Airflow	42
6	Phase 5 – Serving Layer: Dremio & Power BI	46
6.1	Dremio Deployment	46
6.1.1	Connect Dremio to Hive Metastore	47
6.2	Power BI Connection	48
7	Phase 6 – Security: Cloudflare Zero Trust	49
7.1	Cloudflare Operator	49
8	Phase 7 – Observability: Prometheus, Grafana & OpenObserve	52
8.1	kube-prometheus-stack	52
8.2	Fluent Bit – Log Shipping	54
9	Phase 8 – CI/CD: GitHub Actions Pipeline	57
9.1	OIDC Trust Between GitHub and AWS	57
9.2	GitHub Actions Workflows	58
9.3	GitHub Repository Secrets Setup	63
10	Phase 9 – Cost Control: Lambda Node Scheduling	64
11	Phase 10 – ArgoCD GitOps (Optional)	67
11.1	Install ArgoCD	67
12	Network Policies & Security Hardening	69
12.1	Default Deny All	69
13	Ansible Playbooks	71
14	Validation & End-to-End Testing	73
14.1	Full Pipeline Smoke Test	73
14.2	Iceberg Table Verification	74
	Quick Reference Cheat Sheet	76

1 | Prerequisites & Repository Bootstrap

Before writing a single line of Terraform, you must set up your local toolchain, configure cloud credentials, and create the Git repository that will be the single source of truth for all infrastructure.

1.1 Required Tools Installation

❏ Install on your workstation (Windows 10/11)

Every tool below must be at the specified version or newer. Version mismatches are the #1 cause of mysterious Terraform/Helm errors. Run all commands in an **elevated PowerShell** (Run as Administrator). We use **winget** (built into Windows 10 1809+ / Windows 11) as the primary package manager, with **Chocolatey** as a fallback for tools not yet in the winget repository.

```
1 ## Verify winget is available (ships with App Installer on Win10 1809+/Win11)
2 winget --version
3
4 ## Install Chocolatey (fallback package manager)
5 Set-ExecutionPolicy Bypass -Scope Process -Force
6 [System.Net.ServicePointManager]::SecurityProtocol = '
7 [System.Net.ServicePointManager]::SecurityProtocol -bor 3072
8 iex ((New-Object System.Net.WebClient).DownloadString( '
9 'https://community.chocolatey.org/install.ps1'))
10 choco --version
```

Listing 1.1: One-time: enable winget and install Chocolatey (PowerShell as Admin)

```
1 ## AWS CLI v2
2 winget install --id Amazon.AWSCLI -e
3 aws --version # must be >= 2.15
4
5 ## Terraform
6 ## Windows equivalent of tfenv
7 choco install terraform --version=1.7.5 -y
8 ## (Alternative multi-version manager)
9 ## choco install tfswitch -y ; tfswitch 1.7.5
10 terraform version
11
12 ## kubectl
13 winget install --id Kubernetes.kubectl -e
14 kubectl version --client
15
```

```

16 ## Helm 3
17 winget install --id Helm.Helm -e
18 helm version # must be >= 3.14
19
20 ## eksctl
21 choco install eksctl -y
22 eksctl version
23
24 ## tflint (Terraform linter)
25 choco install tflint -y
26 tflint --version
27
28 ## Python + Ansible
29 ## Ansible's control node does NOT run natively on Windows.
30 ## Install it inside WSL2 (Ubuntu) recommended for this project.
31 wsl --install -d Ubuntu
32 ## Then, inside the Ubuntu shell:
33 ## sudo apt update && sudo apt install -y python3-pip
34 ## pip3 install --user ansible==9.5.1 boto3 botocore
35 ## ansible --version
36
37 ## ArgoCD CLI
38 choco install argocd-cli -y
39 argocd version --client
40
41 ## Git (if not already installed)
42 winget install --id Git.Git -e

```

Listing 1.2: Install all required CLI tools (PowerShell as Admin)

>_ Why WSL2 for Ansible?

Ansible's control node officially supports Linux/macOS only. On Windows, install **WSL2 with Ubuntu** (`wsl -install -d Ubuntu`) and run Ansible from there. Managed nodes (the EC2 instances Ansible targets) are unaffected only your local control node needs WSL2.

1.2 AWS Credentials & IAM Setup

Security: Never use root credentials

Create a dedicated IAM user or, better, use an IAM Identity Center (SSO) role for day-to-day operations. The CI/CD pipeline must use OIDC federation (no long-lived keys).

```

1 ## Create a named profile for the project (NOT the default profile)
2 aws configure --profile dataplatform-dev
3 # AWS Access Key ID: <your-key>
4 # AWS Secret Access Key: <your-secret>
5 # Default region: eu-west-1
6 # Default output format: json
7

```

```
8 ## Verify access
9 export AWS_PROFILE=dataplatfom-dev
10 aws sts get-caller-identity
11 # Expected output:
12 # {
13 #   "UserId": "AIDA...",
14 #   "Account": "123456789012",
15 #   "Arn": "arn:aws:iam::123456789012:user/dataplatfom-admin"
16 # }
17
18 ## Set for current shell session
19 export AWS_PROFILE=dataplatfom-dev
20 export AWS_REGION=eu-west-1
```

Listing 1.3: *Configure AWS credentials with named profile*

1.2.1 Minimum IAM Permissions for Terraform

Create an IAM policy named `DataPlatformTerraformPolicy` and attach it to your user/role:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "EKS",
6       "Effect": "Allow",
7       "Action": ["eks:*"],
8       "Resource": "*"
9     },
10    {
11      "Sid": "EC2VPC",
12      "Effect": "Allow",
13      "Action": ["ec2:*", "elasticloadbalancing:*", "autoscaling:*"],
14      "Resource": "*"
15    },
16    {
17      "Sid": "IAM",
18      "Effect": "Allow",
19      "Action": [
20        "iam:CreateRole", "iam:AttachRolePolicy", "iam:PutRolePolicy",
21        "iam:PassRole", "iam:GetRole", "iam:ListRoles",
22        "iam:CreateInstanceProfile", "iam:AddRoleToInstanceProfile",
23        "iam:CreateOpenIDConnectProvider", "iam:GetOpenIDConnectProvider",
24        "iam>DeleteRole", "iam:DetachRolePolicy", "iam:DeleteRolePolicy",
25        "iam:TagRole"
26      ],
27      "Resource": "*"
28    },
29    {
30      "Sid": "S3",
31      "Effect": "Allow",
32      "Action": ["s3:*"],
33      "Resource": "*"
34    }
35  ]
36 }
```

```

34     },
35     {
36         "Sid": "Lambda",
37         "Effect": "Allow",
38         "Action": ["lambda:*", "events:*", "scheduler:*"],
39         "Resource": "*"
40     },
41     {
42         "Sid": "SSM",
43         "Effect": "Allow",
44         "Action": ["ssm:GetParameter", "secretsmanager:*", "kms:*"],
45         "Resource": "*"
46     }
47 ]
48 }

```

Listing 1.4: Required IAM permissions (save as *policy.json*)

```

1 aws iam create-policy \
2   --policy-name DataPlatformTerraformPolicy \
3   --policy-document file://policy.json \
4   --profile dataplatform-dev
5
6 ## Attach to your user
7 aws iam attach-user-policy \
8   --user-name dataplatform-admin \
9   --policy-arn arn:aws:iam::ACCOUNT_ID:policy/DataPlatformTerraformPolicy

```

Listing 1.5: Apply the policy

1.3 Git Repository Structure

Create a **monorepo** with the following layout. Everything lives in one repo for simplicity at MVP scale.

```

1 mkdir data-platform && cd data-platform
2 git init
3 git checkout -b main
4
5 ## Create the full directory tree
6 mkdir -p \
7   terraform/modules/{vpc,eks,iam,s3,lambda-scaling,security-groups} \
8   terraform/environments/{dev,prod} \
9   helm/{minio-operator,minio-tenant,kafka,spark-operator,airflow,\
10     hive-metastore,dremio,monitoring,logging,cloudflare,argocd} \
11   k8s/{namespaces,rbac,network-policies,storage-classes} \
12   ansible/{playbooks,roles,inventories,group_vars} \
13   dags/ \ # Airflow DAGs
14   dbt/{models,macros,tests} \ # dbt project
15   spark-jobs/ \ # PySpark scripts
16   scripts/ \ # Utility bash/python scripts
17   .github/workflows \ # GitHub Actions pipelines
18

```

```
19 ## Essential root files
20 cat > .gitignore << 'EOF'
21 **/.terraform/
22 *.tfstate
23 *.tfstate.backup
24 *.tfvars          # override locally, never commit secrets
25 .terraform.lock.hcl # COMMIT this one (pin provider versions)
26 **/__pycache__/_
27 *.pyc
28 .DS_Store
29 *.egg-info/
30 venv/
31 .env
32 EOF
33
34 cat > .terraform-version << 'EOF'
35 1.7.5
36 EOF
37
38 echo "# Cloud Data Platform" > README.md
39 git add . && git commit -m "chore: initialize monorepo skeleton"
```

Listing 1.6: Create the monorepo skeleton

1.4 Cloudflare R2 – Terraform Remote State Backend

Before any `terraform init`, the R2 bucket must exist.

```
1 ## 1. Log into Cloudflare dashboard -> R2 -> Create bucket
2 ##   Bucket name: terraform-state-dataplatform
3 ##   Location:   Automatic
4
5 ## 2. Create R2 API credentials
6 ##   Dashboard -> R2 -> Manage R2 API tokens
7 ##   Permissions: Object Read & Write (scoped to terraform-state-
8 ##               dataplatform)
9 ##   Save the Access Key ID and Secret Access Key securely
10
11 ## 3. Test connectivity
12 export R2_ACCESS_KEY_ID=<your-r2-access-key>
13 export R2_SECRET_ACCESS_KEY=<your-r2-secret>
14 export R2_ACCOUNT_ID=<your-cloudflare-account-id>
15
16 aws s3 ls s3://terraform-state-dataplatform \
17     --endpoint-url "https://${R2_ACCOUNT_ID}.r2.cloudflarestorage.com" \
18     --region auto \
19     --no-verify-ssl 2>/dev/null && echo "R2 OK" || echo "Check credentials"
```

Listing 1.7: Create Cloudflare R2 bucket for Terraform state

✓ **Checkpoint 0: Prerequisites Done**

- All CLI tools installed and version-verified
- AWS credentials configured with named profile
- Git monorepo initialized with correct structure
- Cloudflare R2 bucket created and accessible

2 | Phase 1 – VPC, Networking & EKS Cluster

2.1 CIDR Design & Subnet Strategy

Getting the CIDR design right at the start avoids painful refactoring later. The design must accommodate:

- 3 Availability Zones (resilience)
- Public subnets (NAT Gateways, future ALBs)
- Private subnets (EKS nodes, RDS, MinIO)
- Intra subnets (internal-only services, no NAT egress)
- Pod IP space via VPC CNI (pods get IPs from the VPC CIDR)

CIDR Sizing Rule of Thumb

EKS with VPC CNI assigns one secondary IP per pod. A `/24` subnet (254 IPs) supports roughly **60–80 pods** after node overhead. Use `/20` for node subnets to leave room for growth.

Table 2.1: Complete CIDR Allocation Plan

Subnet	AZ	CIDR	Purpose
VPC	–	10.0.0.0/16	65,536 IPs total
public-eu-w1a	eu-west-1a	10.0.1.0/24	NAT GW, future ALB
public-eu-w1b	eu-west-1b	10.0.2.0/24	NAT GW, future ALB
public-eu-w1c	eu-west-1c	10.0.3.0/24	NAT GW, future ALB
private-eu-w1a	eu-west-1a	10.0.16.0/20	EKS nodes, pods
private-eu-w1b	eu-west-1b	10.0.32.0/20	EKS nodes, pods
private-eu-w1c	eu-west-1c	10.0.48.0/20	EKS nodes, pods
intra-eu-w1a	eu-west-1a	10.0.64.0/24	RDS, internal DBs
intra-eu-w1b	eu-west-1b	10.0.65.0/24	RDS, internal DBs
intra-eu-w1c	eu-west-1c	10.0.66.0/24	RDS, internal DBs

2.2 Terraform VPC Module

```
1 ## We use the battle-tested terraform-aws-modules/vpc module.
```

```
2  ## It handles IGW, NAT GW, route tables, and tags automatically.
3
4  terraform {
5      required_providers {
6          aws = { source = "hashicorp/aws", version = "~> 5.40" }
7      }
8  }
9
10 module "vpc" {
11     source = "terraform-aws-modules/vpc/aws"
12     version = "5.8.1"
13
14     name = var.name      # e.g. "dataplatfrom-dev"
15     cidr = "10.0.0.0/16"
16
17     ## 3 AZs for high availability
18     azs = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
19
20     ## Subnet CIDRs (order matches AZs above)
21     public_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
22     private_subnets = ["10.0.16.0/20", "10.0.32.0/20", "10.0.48.0/20"]
23     intra_subnets = ["10.0.64.0/24", "10.0.65.0/24", "10.0.66.0/24"]
24
25     ## NAT Gateway: one per AZ for true HA (3 NAT GWs = 3x cost)
26     ## For MVP: single_nat_gateway = true to save ~$100/month
27     enable_nat_gateway = true
28     single_nat_gateway = true # flip to false for prod
29     one_nat_gateway_per_az = false # flip to true for prod
30
31     ## DNS -- required for EKS and Route53
32     enable_dns_hostnames = true
33     enable_dns_support = true
34
35     ## VPC Flow Logs -- required for security compliance
36     enable_flow_log = true
37     create_flow_log_cloudwatch_log_group = true
38     create_flow_log_cloudwatch_iam_role = true
39     flow_log_max_aggregation_interval = 60
40
41     ## Tags required for EKS to discover subnets for load balancers
42     public_subnet_tags = {
43         "kubernetes.io/role/elb" = "1"
44         "kubernetes.io/cluster/${var.cluster_name}" = "shared"
45     }
46     private_subnet_tags = {
47         "kubernetes.io/role/internal-elb" = "1"
48         "kubernetes.io/cluster/${var.cluster_name}" = "shared"
49         "karpenter.sh/discovery" = var.cluster_name
50     }
51
52     tags = merge(var.common_tags, {
53         Module = "vpc"
```

```

54     })
55 }

```

Listing 2.1: *terraform/modules/vpc/main.tf*

```

1 variable "name"          { type = string }
2 variable "cluster_name" { type = string }
3 variable "common_tags"  { type = map(string) default = {} }

```

Listing 2.2: *terraform/modules/vpc/variables.tf*

```

1 output "vpc_id"           { value = module.vpc.vpc_id }
2 output "private_subnet_ids" { value = module.vpc.private_subnets }
3 output "public_subnet_ids" { value = module.vpc.public_subnets }
4 output "intra_subnet_ids"  { value = module.vpc.intra_subnets }
5 output "vpc_cidr_block"    { value = module.vpc.vpc_cidr_block }

```

Listing 2.3: *terraform/modules/vpc/outputs.tf*

2.3 Security Groups

```

1 ## EKS Node Security Group
2 resource "aws_security_group" "eks_nodes" {
3     name           = "${var.cluster_name}-nodes"
4     description    = "EKS worker node security group"
5     vpc_id         = var.vpc_id
6
7     ## Nodes must talk to each other (pod-to-pod, Kubernetes API)
8     ingress {
9         description = "Node to node all ports"
10        from_port   = 0
11        to_port     = 65535
12        protocol    = "tcp"
13        self        = true
14    }
15
16    ## Allow all outbound (required for image pulls, AWS API calls)
17    egress {
18        from_port   = 0
19        to_port     = 0
20        protocol    = "-1"
21        cidr_blocks = ["0.0.0.0/0"]
22    }
23
24    tags = merge(var.common_tags, {
25        Name = "${var.cluster_name}-nodes"
26        "kubernetes.io/cluster/${var.cluster_name}" = "owned"
27    })
28 }
29
30 ## MinIO Security Group
31 resource "aws_security_group" "minio" {
32     name           = "${var.cluster_name}-minio"

```

```

33   description = "MinIO object storage"
34   vpc_id      = var.vpc_id
35
36   ingress {
37     description      = "MinIO API from EKS nodes"
38     from_port        = 9000
39     to_port          = 9001
40     protocol         = "tcp"
41     security_groups  = [aws_security_group.eks_nodes.id]
42   }
43
44   egress {
45     from_port = 0; to_port = 0; protocol = "-1"
46     cidr_blocks = ["0.0.0.0/0"]
47   }
48   tags = merge(var.common_tags, { Name = "${var.cluster_name}-minio" })
49 }
50
51 ## Database Security Group
52 resource "aws_security_group" "databases" {
53   name          = "${var.cluster_name}-databases"
54   description   = "PostgreSQL, SQL Server, Informix"
55   vpc_id        = var.vpc_id
56
57   ingress {
58     description      = "PostgreSQL from EKS"
59     from_port        = 5432; to_port = 5432; protocol = "tcp"
60     security_groups  = [aws_security_group.eks_nodes.id]
61   }
62   ingress {
63     description      = "SQL Server from EKS"
64     from_port        = 1433; to_port = 1433; protocol = "tcp"
65     security_groups  = [aws_security_group.eks_nodes.id]
66   }
67   ingress {
68     description      = "Informix from EKS"
69     from_port        = 9088; to_port = 9088; protocol = "tcp"
70     security_groups  = [aws_security_group.eks_nodes.id]
71   }
72
73   egress {
74     from_port = 0; to_port = 0; protocol = "-1"
75     cidr_blocks = ["0.0.0.0/0"]
76   }
77   tags = merge(var.common_tags, { Name = "${var.cluster_name}-databases" })
78 }

```

Listing 2.4: *terraform/modules/security-groups/main.tf*

2.4 EKS Cluster Terraform Module

```

1 module "eks" {

```

```
2 source = "terraform-aws-modules/eks/aws"
3 version = "20.8.5"
4
5 cluster_name = var.cluster_name
6 cluster_version = "1.29"
7
8 ## Deploy control plane endpoint (private + public for dev)
9 cluster_endpoint_private_access = true
10 cluster_endpoint_public_access = true
11 ## Restrict public access to your office/VPN IP only
12 cluster_endpoint_public_access_cidrs = var.allowed_cidr_blocks
13
14 vpc_id = var.vpc_id
15 subnet_ids = var.private_subnet_ids
16 control_plane_subnet_ids = var.intra_subnet_ids
17
18 ## Enable IRSA (IAM Roles for Service Accounts)
19 enable_irsa = true
20
21 ## EKS Add-ons (managed by AWS, auto-updated)
22 cluster_addons = {
23   coredns = {
24     most_recent = true
25     configuration_values = jsonencode({
26       replicaCount = 2
27     })
28   }
29   kube-proxy = { most_recent = true }
30   vpc-cni = { most_recent = true } # AWS VPC CNI for pod
    networking
31   aws-ebs-csi-driver = { most_recent = true
32     service_account_role_arn = module.ebs_csi_irsa.iam_role_arn
33   }
34 }
35
36 ## Node Groups
37
38 ## System node group: always-on, runs Kube system + operators
39 eks_managed_node_groups = {
40
41   system = {
42     name = "system"
43     instance_types = ["t3.medium"]
44     min_size = 2
45     max_size = 4
46     desired_size = 2
47     capacity_type = "ON_DEMAND"
48     disk_size = 50
49
50     labels = {
51       "node-role" = "system"
52     }
53   }
54 }
```

```
53     taints = [  
54         {  
55             key      = "node-role"  
56             value    = "system"  
57             effect    = "NO_SCHEDULE"  
58         }  
59     ]  
60  
61     tags = { "node-group" = "system" }  
62 }  
63  
64 ## Compute node group: scaled by Lambda 8AM-6PM  
65 compute = {  
66     name           = "compute"  
67     instance_types = ["m5.xlarge"]    # 4 vCPU, 16GB -- good for Spark  
68     min_size       = 0  
69     max_size       = 10  
70     desired_size   = 3  
71     capacity_type  = "SPOT"           # 60-80% cheaper for batch  
72     workloads  
73  
74     labels = {  
75         "node-role"      = "compute"  
76         "workload-type"  = "spark"  
77     }  
78  
79     block_device_mappings = {  
80         xvda = {  
81             device_name = "/dev/xvda"  
82             ebs = {  
83                 volume_size      = 100  
84                 volume_type      = "gp3"  
85                 iops              = 3000  
86                 throughput        = 125  
87                 delete_on_termination = true  
88             }  
89         }  
90     }  
91  
92     tags = {  
93         "node-group" = "compute"  
94         ## Tag required for Lambda scheduling  
95         "auto-scaling-schedule" = "business-hours"  
96     }  
97 }  
98 ## Storage node group: always-on for MinIO (needs stable IP + persistent  
99 EBS)  
100 storage = {  
101     name           = "storage"  
102     instance_types = ["m5.2xlarge"] # 8 vCPU, 32GB -- MinIO needs memory  
103     min_size       = 1              # Keep 1 running 24/7 for MinIO
```

```
103     max_size           = 4
104     desired_size       = 4                               # 4 pods = 1 per server in MinIO pool
105
106     labels = {
107         "node-role"      = "storage"
108         "minio-node"     = "true"
109     }
110
111     block_device_mappings = {
112         xvda = {
113             device_name = "/dev/xvda"
114             ebs = {
115                 volume_size = 50
116                 volume_type = "gp3"
117                 delete_on_termination = true
118             }
119         }
120     }
121
122     tags = { "node-group" = "storage" }
123 }
124
125
126 ## Cluster access: grant admin to the Terraform executor
127 manage_aws_auth_configmap = true
128 aws_auth_roles = [
129     {
130         rolearn    = var.admin_role_arn
131         username   = "cluster-admin"
132         groups     = ["system:masters"]
133     }
134 ]
135
136 tags = var.common_tags
137 }
138
139 ## IRSA for EBS CSI Driver
140 module "ebs_csi_irsa" {
141     source = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-accounts-eks"
142     version = "5.39.1"
143
144     role_name           = "${var.cluster_name}-ebs-csi"
145     attach_ebs_csi_policy = true
146
147     oidc_providers = {
148         main = {
149             provider_arn          = module.eks.oidc_provider_arn
150             namespace_service_accounts = ["kube-system:ebs-csi-controller-sa"]
151         }
152     }
153 }
```


153

}

Listing 2.5: *terraform/modules/eks/main.tf*

2.5 Environment Root Configuration

```

1 terraform {
2   required_version = ">= 1.7.0"
3
4   ## Cloudflare R2 remote state backend
5   backend "s3" {
6     bucket          = "terraform-state-dataplatform"
7     key             = "dev/terraform.tfstate"
8     region          = "auto"
9     endpoint        = "https://${R2_ACCOUNT_ID}.r2.
10                      cloudflarestorage.com"
11     skip_credentials_validation = true
12     skip_metadata_api_check    = true
13     skip_region_validation     = true
14     force_path_style           = true
15   }
16
17   required_providers {
18     aws      = { source = "hashicorp/aws",      version = "~> 5.40" }
19     kubernetes = { source = "hashicorp/kubernetes", version = "~> 2.27" }
20     helm      = { source = "hashicorp/helm",     version = "~> 2.13" }
21   }
22 }
23
24 provider "aws" {
25   region = var.aws_region
26   profile = "dataplat-form-dev"
27
28   default_tags {
29     tags = {
30       Project      = "data-platform"
31       Environment  = "dev"
32       ManagedBy    = "terraform"
33       Owner        = "data-engineering"
34     }
35   }
36 }
37
38 module "vpc" {
39   source      = "../modules/vpc"
40   name        = "dataplat-form-dev"
41   cluster_name = "dataplat-form-dev"
42 }
43
44 module "security_groups" {
45   source      = "../modules/security-groups"
46   cluster_name = "dataplat-form-dev"

```

```
46     vpc_id      = module.vpc.vpc_id
47     common_tags = {}
48 }
49
50 module "iam" {
51     source      = "../../modules/iam"
52     cluster_name = "dataplatfom-dev"
53     oidc_provider_arn = module.eks.oidc_provider_arn
54     oidc_provider_url = module.eks.cluster_oidc_issuer_url
55 }
56
57 module "eks" {
58     source      = "../../modules/eks"
59     cluster_name = "dataplatfom-dev"
60     vpc_id      = module.vpc.vpc_id
61     private_subnet_ids = module.vpc.private_subnet_ids
62     intra_subnet_ids  = module.vpc.intra_subnet_ids
63     allowed_cidr_blocks = ["YOUR.OFFICE.IP.CIDR/32"]
64     admin_role_arn     = module.iam.admin_role_arn
65 }
66
67 module "s3" {
68     source      = "../../modules/s3"
69     cluster_name = "dataplatfom-dev"
70     aws_region  = var.aws_region
71 }
72
73 module "lambda_scaling" {
74     source      = "../../modules/lambda-scaling"
75     cluster_name = "dataplatfom-dev"
76     aws_region  = var.aws_region
77     scale_up_cron = "0 7 * * ? *" # 8AM UTC+1 = 7AM UTC
78     scale_down_cron = "0 17 * * ? *" # 6PM UTC+1 = 5PM UTC
79     nodegroup_name = "compute"
80     min_size_up    = 2
81     desired_up     = 3
82 }
```

Listing 2.6: *terraform/environments/dev/main.tf*

```
1 aws_region = "eu-west-1"
2 ## NOTE: Never commit secrets here. Use environment variables or
3 ## AWS Secrets Manager for sensitive values.
```

Listing 2.7: *terraform/environments/dev/terraform.tfvars*

2.6 Deploying Phase 1 Infrastructure

```
1 cd terraform/environments/dev
2
3 ## Initialize -- downloads providers, configures R2 backend
4 ## Pass R2 credentials as env vars (NOT in tfvars)
5 export AWS_ACCESS_KEY_ID="$R2_ACCESS_KEY_ID"
```

```
6 export AWS_SECRET_ACCESS_KEY="$R2_SECRET_ACCESS_KEY"
7 terraform init
8
9 ## Restore your AWS profile for actual provisioning
10 export AWS_PROFILE=dataplatfom-dev
11 unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
12
13 ## Lint check
14 tflint --init
15 tflint
16
17 ## Format check
18 terraform fmt -check -recursive
19
20 ## Preview changes -- READ THE OUTPUT CAREFULLY
21 terraform plan -out=tfplan
22
23 ## Apply (creates ~45-50 resources, takes 15-20 minutes)
24 terraform apply tfplan
25
26 ## Configure kubectl to point at the new cluster
27 aws eks update-kubeconfig \
28     --region eu-west-1 \
29     --name dataplatfom-dev \
30     --profile dataplatfom-dev
31
32 ## Verify nodes are running
33 kubectl get nodes -o wide
34 kubectl get pods -n kube-system
35
36 ## Verify EBS CSI driver
37 kubectl get pods -n kube-system -l app=ebs-csi-controller
```

Listing 2.8: Phase 1 deployment commands

2.6.1 Create gp3 Storage Class

```
1 apiVersion: storage.k8s.io/v1
2 kind: StorageClass
3 metadata:
4   name: gp3
5   annotations:
6     storageclass.kubernetes.io/is-default-class: "true" # Make it default
7 provisioner: ebs.csi.aws.com
8 volumeBindingMode: WaitForFirstConsumer # Provision EBS in the same AZ as
    the pod
9 allowVolumeExpansion: true
10 parameters:
11   type: gp3
12   iops: "3000"
13   throughput: "125"
14   encrypted: "true"
15   kmsKeyId: "" # Leave empty to use AWS-managed key
```

```
16 reclaimPolicy: Retain # IMPORTANT: Don't auto-delete EBS when PVC is
    deleted
```

Listing 2.9: *k8s/storage-classes/gp3.yaml*

```
1 kubectl apply -f k8s/storage-classes/gp3.yaml
2
3 ## Remove default annotation from old gp2 storage class
4 kubectl patch storageclass gp2 \
5   -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-
6     class": "false"}}}'
7
8 ## Verify
9 kubectl get storageclass
```

Listing 2.10: *Apply storage class and remove old gp2 default*

2.7 Kubernetes Namespace & RBAC Setup

```
1 ---
2 apiVersion: v1
3 kind: Namespace
4 metadata:
5   name: platform-ingestion
6   labels:
7     team: data-engineering
8     env: dev
9 ---
10 apiVersion: v1
11 kind: Namespace
12 metadata:
13   name: platform-storage
14   labels:
15     team: infra
16     env: dev
17 ---
18 apiVersion: v1
19 kind: Namespace
20 metadata:
21   name: platform-metastore
22   labels:
23     team: data-engineering
24     env: dev
25 ---
26 apiVersion: v1
27 kind: Namespace
28 metadata:
29   name: platform-compute
30   labels:
31     team: data-engineering
32     env: dev
33 ---
34 apiVersion: v1
```

```
35 kind: Namespace
36 metadata:
37   name: platform-orchestr
38   labels:
39     team: data-engineering
40     env: dev
41 ---
42 apiVersion: v1
43 kind: Namespace
44 metadata:
45   name: platform-serving
46   labels:
47     team: analytics
48     env: dev
49 ---
50 apiVersion: v1
51 kind: Namespace
52 metadata:
53   name: platform-monitoring
54   labels:
55     team: sre
56     env: dev
57 ---
58 apiVersion: v1
59 kind: Namespace
60 metadata:
61   name: platform-logging
62   labels:
63     team: sre
64     env: dev
65 ---
66 apiVersion: v1
67 kind: Namespace
68 metadata:
69   name: platform-security
70   labels:
71     team: infra
72     env: dev
```

Listing 2.11: *k8s/namespaces/namespaces.yaml*

```
1 kubectl apply -f k8s/namespaces/namespaces.yaml
2 kubectl get namespaces | grep platform
```

Listing 2.12: *Apply all namespaces at once*

✔ **Checkpoint 1: Phase 1 Complete**

- VPC with 3 AZs, 9 subnets, NAT Gateway deployed
- EKS 1.29 with 3 node groups (system, compute, storage)
- EBS CSI driver with gp3 as default StorageClass
- All 9 platform namespaces created
- kubectl configured and nodes verified Ready

3 | Phase 2 – Storage Layer: MinIO, Hive & Databases

3.1 cert-manager (TLS Prerequisites)

MinIO's operator requires cert-manager for TLS certificate management. Install it first.

```
1 helm repo add jetstack https://charts.jetstack.io
2 helm repo update
3
4 helm install cert-manager jetstack/cert-manager \
5   --namespace cert-manager \
6   --create-namespace \
7   --version v1.14.4 \
8   --set installCRDs=true \
9   --set global.leaderElection.namespace=cert-manager
10
11 ## Wait for cert-manager pods
12 kubectl rollout status deploy/cert-manager -n cert-manager
13 kubectl rollout status deploy/cert-manager-webhook -n cert-manager
14
15 ## Create a self-signed ClusterIssuer for internal services
16 cat <<'EOF' | kubectl apply -f -
17 apiVersion: cert-manager.io/v1
18 kind: ClusterIssuer
19 metadata:
20   name: selfsigned-issuer
21 spec:
22   selfSigned: {}
23 EOF
```

Listing 3.1: *Install cert-manager*

3.2 MinIO Operator Installation

```
1 operator:
2   replicaCount: 2
3   resources:
4     requests:
5       cpu: "200m"
6       memory: "256Mi"
7     limits:
8       cpu: "500m"
9       memory: "512Mi"
10  ## Operator pod must run on system nodes
```

```

11   nodeSelector:
12     node-role: system
13   tolerations:
14     - key: "node-role"
15       operator: "Equal"
16       value: "system"
17       effect: "NoSchedule"

```

Listing 3.2: *helm/minio-operator/values.yaml*

```

1 helm repo add minio-operator https://operator.min.io
2 helm repo update
3
4 helm install minio-operator minio-operator/operator \
5   --namespace minio-operator \
6   --create-namespace \
7   --version 6.0.3 \
8   --values helm/minio-operator/values.yaml
9
10 ## Wait for operator
11 kubectl rollout status deploy/minio-operator -n minio-operator
12
13 ## Verify CRDs are installed
14 kubectl get crd | grep minio
15 # tenants.minio.min.io <-- this must exist

```

Listing 3.3: *Install MinIO Operator*

3.3 MinIO Tenant (Lakehouse Storage)

3.3.1 Create MinIO Credentials Secret

```

1 ## Generate a strong password
2 MINIO_ROOT_USER="minioadmin"
3 MINIO_ROOT_PASSWORD=$(openssl rand -base64 32)
4
5 ## Store in Kubernetes secret
6 kubectl create secret generic minio-env-config \
7   --namespace platform-storage \
8   --from-literal=config.env="export MINIO_ROOT_USER=\"${MINIO_ROOT_USER}\"
9 export MINIO_ROOT_PASSWORD=\"${MINIO_ROOT_PASSWORD}\"
10
11 ## Also store in AWS Secrets Manager for reference
12 aws secretsmanager create-secret \
13   --name "datapatform/dev/minio-credentials" \
14   --secret-string "{\"username\":\"${MINIO_ROOT_USER}\",\"password\":\"${
15     MINIO_ROOT_PASSWORD}\"}" \
16   --region eu-west-1 \
17   --profile datapatform-dev
18 echo "MinIO password (save this): $MINIO_ROOT_PASSWORD"

```

Listing 3.4: *Create MinIO admin credentials secret*


```

1 apiVersion: minio.min.io/v2
2 kind: Tenant
3 metadata:
4   name: lakehouse
5   namespace: platform-storage
6   labels:
7     app: minio-lakehouse
8 spec:
9   image: "minio/minio:RELEASE.2024-03-15T01-07-19Z"
10  imagePullPolicy: IfNotPresent
11
12  ## Reference to credentials secret created above
13  configuration:
14    name: minio-env-config
15
16  ## 4 server pods, each with 4 x 500Gi EBS gp3 volumes
17  ## Total raw capacity: 4 * 4 * 500Gi = 8 TiB
18  ## Erasure coding (EC:4) provides ~4 TiB usable with N/2 parity
19  pools:
20    - name: pool-0
21      servers: 4
22      volumesPerServer: 4
23      volumeClaimTemplate:
24        metadata:
25          name: data
26        spec:
27          accessModes:
28            - ReadWriteOnce
29          storageClassName: gp3
30          resources:
31            requests:
32              storage: 500Gi
33          ## Pin MinIO pods to the storage node group
34          nodeSelector:
35            node-role: storage
36          tolerations: []          # storage nodes have no taints
37          resources:
38            requests:
39              cpu: "2"
40              memory: "8Gi"
41            limits:
42              cpu: "4"
43              memory: "16Gi"
44
45  ## S3 API and Console ports
46  mountPath: /export
47
48  ## TLS via cert-manager
49  requestAutoCert: true
50
51  ## Expose MinIO API within cluster only
52  exposeServices:

```

```

53     minio: false      # Don't create LoadBalancer -- use ClusterIP
54     console: false    # Console exposed via Cloudflare tunnel
55
56     ## Pre-create buckets
57     buckets:
58       - name: lakehouse      # Bronze/Silver/Gold Iceberg tables
59       - name: landing        # Raw file uploads (CSV, XLSX)
60       - name: backups        # CloudNativePG backups
61       - name: logs           # Fluent Bit log output
62       - name: spark-jobs     # Spark job scripts
63       - name: dbt-artifacts   # dbt compiled artifacts
64
65     ## Pre-create service accounts
66     users:
67       - name: minio-env-config # Admin user from secret above
68
69     ## Enable versioning on critical buckets (data protection)
70     features:
71       bucketDNS: false
72
73     ## Prometheus metrics endpoint
74     prometheusOperator: false # We'll use ServiceMonitor manually
75
76     ## Console access (for debugging -- accessed via Cloudflare tunnel)
77     console:
78       image: "minio/console:v0.35.0"

```

Listing 3.5: *helm/minio-tenant/tenant.yaml* – Full MinIO Tenant CRD

```

1 kubectl apply -f helm/minio-tenant/tenant.yaml
2
3 ## Watch tenant status (takes 5-10 minutes for EBS volumes to provision)
4 kubectl get tenant lakehouse -n platform-storage -w
5
6 ## Once status shows "Initialized":
7 kubectl get pods -n platform-storage
8 ## Expected: 4 pods named minio-pool-0-{0,1,2,3}
9
10 ## Get MinIO service endpoint
11 kubectl get svc -n platform-storage
12 ## minio.platform-storage.svc.cluster.local:9000 <-- S3 API endpoint
13
14 ## Port-forward to test locally
15 kubectl port-forward svc/minio -n platform-storage 9000:9000 &
16 mc alias set lakehouse http://localhost:9000 $MINIO_ROOT_USER
17   $MINIO_ROOT_PASSWORD
18 mc ls lakehouse
19 ## Should show: lakehouse landing backups logs spark-jobs dbt-artifacts

```

Listing 3.6: *Apply MinIO tenant and verify*

3.4 PostgreSQL via CloudNativePG

```

1 helm repo add cnpg https://cloudnative-pg.github.io/charts
2 helm repo update
3
4 helm install cnpg cnpg/cloudnative-pg \
5   --namespace cnpg-system \
6   --create-namespace \
7   --version 0.20.0
8
9 kubectl rollout status deploy/cnpg-controller-manager -n cnpg-system

```

Listing 3.7: Install CloudNativePG operator

```

1 ## This cluster backs the Hive Metastore (catalog for Iceberg)
2 apiVersion: postgresql.cnpg.io/v1
3 kind: Cluster
4 metadata:
5   name: pg-hms
6   namespace: platform-metastore
7 spec:
8   instances: 3
9   primaryUpdateStrategy: unsupervised
10
11   postgresql:
12     parameters:
13       shared_buffers: "256MB"
14       max_connections: "200"
15       work_mem: "4MB"
16
17   storage:
18     size: 50Gi
19     storageClass: gp3
20
21   bootstrap:
22     initdb:
23       database: hive_metastore
24       owner: hive
25       secret:
26         name: pg-hms-user-secret # Created below
27
28   backup:
29     retentionPolicy: "7d"
30     barmanObjectStore:
31       destinationPath: "s3://backups/cnpg/hms/"
32       endpointURL: "http://minio.platform-storage.svc.cluster.local:9000"
33       s3Credentials:
34         accessKeyId:
35           name: minio-s3-credentials
36           key: ACCESS_KEY
37         secretAccessKey:
38           name: minio-s3-credentials
39           key: SECRET_KEY
40
41   nodeSelector:

```

```
42 node-role: storage
```

Listing 3.8: *k8s/databases/postgres-hms.yaml – PostgreSQL for Hive Metastore*

```
1 ## Create HMS user secret
2 kubectl create secret generic pg-hms-user-secret \
3   --namespace platform-metastore \
4   --from-literal=username=hive \
5   --from-literal=password=$(openssl rand -base64 24)
6
7 ## Create Airflow metadata DB secret
8 kubectl create secret generic pg-airflow-user-secret \
9   --namespace platform-orchestr \
10  --from-literal=username=airflow \
11  --from-literal=password=$(openssl rand -base64 24)
12
13 ## Deploy HMS PostgreSQL
14 kubectl apply -f k8s/databases/postgres-hms.yaml
15
16 ## Watch it come up (3 pods: 1 primary + 2 replicas)
17 kubectl get cluster pg-hms -n platform-metastore -w
18
19 ## Get the read-write service (points to primary)
20 kubectl get svc -n platform-metastore | grep rw
21 # pg-hms-rw ClusterIP ... 5432/TCP
```

Listing 3.9: *Create secrets and deploy PostgreSQL clusters*

3.5 Hive Metastore Deployment

```
1 image:
2   repository: apache/hive
3   tag: "4.0.0"
4   pullPolicy: IfNotPresent
5
6 replicaCount: 2      # 2 HMS instances for HA
7
8 service:
9   type: ClusterIP
10  port: 9083          # Thrift port used by Spark, Dremio, dbt
11
12 ## PostgreSQL connection (uses CloudNativePG rw service)
13 database:
14   host: pg-hms-rw.platform-metastore.svc.cluster.local
15   port: 5432
16   name: hive_metastore
17   user: hive
18   existingSecret: pg-hms-user-secret
19   secretKey: password
20
21 ## MinIO warehouse location
22 warehouseDir: "s3a://lakehouse/warehouse/"
23
```

```

24 ## MinIO S3A configuration
25 s3:
26   endpoint: "http://minio.platform-storage.svc.cluster.local:9000"
27   pathStyleAccess: true
28   existingSecret: minio-s3-credentials
29   accessKeyRef: ACCESS_KEY
30   secretKeyRef: SECRET_KEY
31
32 ## Hadoop S3A client configuration (hive-site.xml)
33 extraConfig:
34   "fs.s3a.connection.maximum": "100"
35   "fs.s3a.connection.timeout": "200000"
36   "fs.s3a.attempts.maximum": "20"
37   "iceberg.engine.hive.lock-enabled": "false"
38
39 resources:
40   requests:
41     cpu: "500m"
42     memory: "1Gi"
43   limits:
44     cpu: "2"
45     memory: "4Gi"
46
47 nodeSelector:
48   node-role: compute

```

Listing 3.10: `helm/hive-metastore/values.yaml`

```

1 ## Create a MinIO service account with policy (read/write on lakehouse
  bucket)
2 kubectl port-forward svc/minio -n platform-storage 9000:9000 &
3
4 mc admin user add lakehouse hive-sa $(openssl rand -base64 16)
5 mc admin policy attach lakehouse readwrite --user hive-sa
6 ## Note the generated credentials for the secret below
7
8 kubectl create secret generic minio-s3-credentials \
9   --namespace platform-metastore \
10  --from-literal=ACCESS_KEY=hive-sa \
11  --from-literal=SECRET_KEY="<generated-secret>"

```

Listing 3.11: Create MinIO service account for S3A access

```

1 ## Add custom Hive Metastore Helm chart (community chart)
2 helm repo add gradient https://gradient.github.io/bigdata-charts/
3 helm repo update
4
5 helm install hive-metastore gradient/hive \
6   --namespace platform-metastore \
7   --version 1.0.0 \
8   --values helm/hive-metastore/values.yaml
9
10 ## Verify HMS is running and can connect to PostgreSQL

```

```
11 kubectl logs -n platform-metastore -l app=hive --follow
12 ## Look for: "Starting Hive Metastore Server"
13
14 ## Test Thrift connection from a debug pod
15 kubectl run -it --rm debug --image=apache/hive:4.0.0 \
16   --namespace platform-metastore -- \
17   /opt/hive/bin/hive --service metatool -listFSRoot
```

Listing 3.12: Deploy Hive Metastore

✓ **Checkpoint 2: Phase 2 Complete**

- cert-manager deployed with self-signed ClusterIssuer
- MinIO operator running; Lakehouse tenant with 8TiB raw capacity
- All 6 buckets created and verified
- CloudNativePG operator; 3-instance PostgreSQL for Hive Metastore
- Hive Metastore 4.0 running; Thrift on port 9083

4 | Phase 3 – Ingestion: Kafka, NiFi & CDC

4.1 Strimzi Kafka Operator

```
1 helm repo add strimzi https://strimzi.io/charts/
2 helm repo update
3
4 helm install strimzi strimzi/strimzi-kafka-operator \
5   --namespace platform-ingestion \
6   --version 0.40.0 \
7   --set watchNamespaces="{platform-ingestion}"
8
9 kubectl rollout status deploy/strimzi-cluster-operator -n platform-ingestion
10 kubectl get crd | grep kafka
11 ## Should show: kafkas.kafka.strimzi.io, kafkatopics.kafka.strimzi.io, etc.
```

Listing 4.1: Install Strimzi Operator

```
1 apiVersion: kafka.strimzi.io/v1beta2
2 kind: Kafka
3 metadata:
4   name: platform
5   namespace: platform-ingestion
6 spec:
7   kafka:
8     version: 3.7.0
9     replicas: 3
10    listeners:
11      - name: plain
12        port: 9092
13        type: internal
14        tls: false
15      - name: tls
16        port: 9093
17        type: internal
18        tls: true
19        authentication:
20          type: scram-sha-512          # Client authentication
21    authorization:
22      type: simple                    # ACL-based authorization
23
24    config:
25      ## Replication and durability
26      offsets.topic.replication.factor: 3
27      transaction.state.log.replication.factor: 3
28      transaction.state.log.min.isr: 2
```

```
29     default.replication.factor: 3
30     min.insync.replicas: 2
31
32     ## Retention: 7 days (allows pipeline replay)
33     log.retention.hours: 168
34     log.retention.bytes: 107374182400    # 100GB per partition max
35
36     ## Compression
37     compression.type: zstd
38
39     storage:
40       type: jbod
41       volumes:
42         - id: 0
43           type: persistent-claim
44           size: 200Gi
45           class: gp3
46           deleteClaim: false    # Keep data if pod is deleted
47
48     resources:
49       requests:
50         cpu: "1"
51         memory: "4Gi"
52       limits:
53         cpu: "4"
54         memory: "8Gi"
55
56     nodeSelector:
57       node-role: compute
58
59     ## Prometheus JMX metrics
60     metricsConfig:
61       type: jmxPrometheusExporter
62       valueFrom:
63         configMapKeyRef:
64           name: kafka-metrics
65           key: kafka-metrics-config.yml
66
67     zookeeper:
68       replicas: 3
69       storage:
70         type: persistent-claim
71         size: 50Gi
72         class: gp3
73         deleteClaim: false
74       resources:
75         requests:
76           cpu: "500m"
77           memory: "1Gi"
78       nodeSelector:
79         node-role: compute
80
```



```

81 entityOperator:
82   topicOperator: {}      # Manages KafkaTopic CRDs
83   userOperator: {}      # Manages KafkaUser CRDs (credentials)

```

Listing 4.2: *k8s/kafka/kafka-cluster.yaml*

```

1  ## Apply the Kafka cluster
2  kubectl apply -f k8s/kafka/kafka-cluster.yaml
3  kubectl wait kafka/platform --for=condition=Ready --timeout=10m -n platform-
   ingestion
4
5  ## Create topics via KafkaTopic CRDs
6  cat <<'EOF' | kubectl apply -f -
7  ---
8  apiVersion: kafka.strimzi.io/v1beta2
9  kind: KafkaTopic
10 metadata:
11   name: bronze-pg-public-agency-budget
12   namespace: platform-ingestion
13   labels:
14     strimzi.io/cluster: platform
15 spec:
16   partitions: 6          # Parallelism = Spark executor count
17   replicas: 3
18   config:
19     retention.ms: "604800000" # 7 days
20     cleanup.policy: "delete"
21     compression.type: "zstd"
22 ---
23 apiVersion: kafka.strimzi.io/v1beta2
24 kind: KafkaTopic
25 metadata:
26   name: bronze-files-landing
27   namespace: platform-ingestion
28   labels:
29     strimzi.io/cluster: platform
30 spec:
31   partitions: 3
32   replicas: 3
33   config:
34     retention.ms: "86400000" # 1 day (files ingested quickly)
35 EOF

```

Listing 4.3: *Create Kafka topics for each data layer*

4.2 Apache NiFi Deployment

```

1  ## Using the apache/nifi Helm chart
2  replicaCount: 1          # MVP: single instance; scale to 3 for HA
3
4  image:
5    repository: apache/nifi
6    tag: "1.26.0"

```

```

7
8 ## NiFi requires significant memory for flow processing
9 resources:
10   requests:
11     cpu: "2"
12     memory: "4Gi"
13   limits:
14     cpu: "4"
15     memory: "8Gi"
16
17 ## Persistent storage for NiFi data (flowfile repo, provenance, content repo)
18 persistence:
19   enabled: true
20   storageClass: gp3
21   subPath:
22     flowfileRepoStorage:
23       size: 50Gi
24     contentRepoStorage:
25       size: 100Gi
26     provenanceRepoStorage:
27       size: 50Gi
28     databaseRepoStorage:
29       size: 10Gi
30
31 ## NiFi will connect to these Kafka brokers
32 zookeeper:
33   enabled: false # Use Strimzi's ZooKeeper, not NiFi's built-in
34
35 ## Authentication
36 auth:
37   enabled: true
38   ## Use single-user auth for MVP (LDAP for production)
39   singleUser:
40     username: nifi-admin
41     password: "" # Set via --set auth.singleUser.password=... in
42                   Helm install
43
44 ## Bootstrap NiFi with custom properties
45 properties:
46   sensitiveKey: "nifi-sensitive-key-change-me-32chars"
47   webProxyHost: "nifi.dataplatform.example.com" # Cloudflare tunnel FQDN
48
49 nodeSelector:
50   node-role: compute

```

Listing 4.4: *helm/nifi/values.yaml*

```

1 helm repo add apache-nifi https://cetic.github.io/helm-charts
2 helm repo update
3
4 NIFI_PASSWORD=$(openssl rand -base64 16)
5 kubectl create secret generic nifi-single-user-creds \

```

```

6  --namespace platform-ingestion \
7  --from-literal=username=nifi-admin \
8  --from-literal=password="${NIFI_PASSWORD}"
9
10 helm install nifi apache-nifi/nifi \
11  --namespace platform-ingestion \
12  --version 1.4.0 \
13  --values helm/nifi/values.yaml \
14  --set auth.singleUser.password="${NIFI_PASSWORD}"
15
16 echo "NiFi password: $NIFI_PASSWORD"
17
18 ## Wait for NiFi (takes 3-5 minutes to bootstrap)
19 kubectl rollout status statefulset/nifi -n platform-ingestion --timeout=10m
20
21 ## Port-forward to verify the UI (before Cloudflare tunnel is set up)
22 kubectl port-forward svc/nifi -n platform-ingestion 8443:8443 &
23 ## Access: https://localhost:8443/nifi

```

Listing 4.5: *Deploy NiFi*

4.2.1 Configuring the CDC Flow in NiFi

Once NiFi is accessible, configure the CDC ingestion flow via the UI or NiFi Registry. The logical flow for PostgreSQL CDC:

☰ NiFi CDC Flow: PostgreSQL to Kafka

1. CaptureChangePostgreSQL processor:

- JDBC URL: `jdbc:postgresql://pg-source-rw.platform-storage:5432/source_db`
- Replication Slot: `nifi_cdc_slot`
- Publication: `nifi_publication` (created in PostgreSQL)
- Output Format: Avro

2. PublishKafkaRecord__2__6 processor:

- Kafka Brokers: `platform-kafka-bootstrap.platform-ingestion:9092`
- Topic: `bronze.${source.schema}.${source.table}`
- Record Writer: AvroRecordSetWriter

```

1  -- Connect to source_db as superuser (nifi_user must have REPLICATION role)
2  ALTER USER nifi_user REPLICATION;
3
4  -- Create publication for all tables you want to capture
5  CREATE PUBLICATION nifi_publication FOR TABLE
6    agency_budget,
7    agency_master,
8    client_accounts;
9
10 -- Create replication slot (NiFi will use this)
11 SELECT pg_create_logical_replication_slot('nifi_cdc_slot', 'pgoutput');
12

```

```
13 -- Verify slot is created
14 SELECT slot_name, plugin, slot_type, active FROM pg_replication_slots;
```

Listing 4.6: Set up PostgreSQL for logical replication (run on the source DB)

✓ **Checkpoint 3: Phase 3 Complete**

- Strimzi Kafka 3-broker cluster with SCRAM auth and ACLs
- All Kafka topics created (bronze.*, file ingestion)
- NiFi deployed and accessible; CDC flow configured
- PostgreSQL publication and replication slot created

5 | Phase 4 – Compute: Spark, dbt & Airflow

5.1 Spark Operator

```
1 helm repo add spark-operator https://kubeflow.github.io/spark-operator
2 helm repo update
3
4 helm install spark-operator spark-operator/spark-operator \
5   --namespace platform-compute \
6   --version 1.1.27 \
7   --set webhook.enable=true \
8   --set webhook.port=8080 \
9   --set metrics.enable=true \
10  --set metrics.port=10254 \
11  --set metrics.endpoint=/metrics \
12  --set nodeSelector."node-role"=system \
13  --set tolerations[0].key=node-role \
14  --set tolerations[0].value=system \
15  --set tolerations[0].effect=NoSchedule
16
17 kubectl rollout status deploy/spark-operator -n platform-compute
18 kubectl get crd | grep spark
19 ## sparkapplications.sparkoperator.k8s.io <-- must exist
```

Listing 5.1: Install Kubeflow Spark Operator

5.1.1 Spark RBAC & Service Account

```
1 ---
2 apiVersion: v1
3 kind: ServiceAccount
4 metadata:
5   name: spark-sa
6   namespace: platform-compute
7 ---
8 apiVersion: rbac.authorization.k8s.io/v1
9 kind: Role
10 metadata:
11   name: spark-role
12   namespace: platform-compute
13 rules:
14   - apiGroups: [""]
15     resources: ["pods", "services", "configmaps", "persistentvolumeclaims"]
16     verbs: ["create", "delete", "get", "list", "watch", "patch", "update"]
17   - apiGroups: [""]
18     resources: ["pods/log"]
```

```

19     verbs: ["get", "list"]
20 ---
21 apiVersion: rbac.authorization.k8s.io/v1
22 kind: RoleBinding
23 metadata:
24   name: spark-rolebinding
25   namespace: platform-compute
26 roleRef:
27   apiGroup: rbac.authorization.k8s.io
28   kind: Role
29   name: spark-role
30 subjects:
31   - kind: ServiceAccount
32     name: spark-sa
33     namespace: platform-compute

```

Listing 5.2: *k8s/rbac/spark-rbac.yaml*

5.1.2 Spark Docker Image with Iceberg

```

1 FROM apache/spark:3.5.1-scala2.12-java17-python3-ubuntu
2
3 USER root
4
5 ## Download Iceberg + AWS S3A JARs
6 RUN wget -q -P /opt/spark/jars/ \
7   https://repo1.maven.org/maven2/org/apache/iceberg/iceberg-spark-runtime
8   -3.5_2.12/1.5.0/iceberg-spark-runtime-3.5_2.12-1.5.0.jar \
9   https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.3.4/hadoop-
10   aws-3.3.4.jar \
11   https://repo1.maven.org/maven2/com/amazonaws/aws-java-sdk-bundle/1.12.262/
12   aws-java-sdk-bundle-1.12.262.jar \
13   https://repo1.maven.org/maven2/com/crealytics/spark-excel_2.12/3.5.0_0
14   .20.3/spark-excel_2.12-3.5.0_0.20.3.jar
15
16 ## Python dependencies
17 COPY requirements.txt /tmp/
18 RUN pip3 install --no-cache-dir -r /tmp/requirements.txt
19
20 USER 185 # Spark default non-root user

```

Listing 5.3: *spark-jobs/Dockerfile – Custom Spark + Iceberg image*

```

1 ## Build and push to your container registry (e.g., ECR)
2 AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
3 ECR_REPO="${AWS_ACCOUNT_ID}.dkr.ecr.eu-west-1.amazonaws.com/spark-iceberg"
4
5 ## Create ECR repo
6 aws ecr create-repository --repository-name spark-iceberg --region eu-west-1
7
8 ## Authenticate Docker to ECR
9 aws ecr get-login-password --region eu-west-1 | \
10  docker login --username AWS --password-stdin ${ECR_REPO}

```

```

11
12 ## Build and push
13 docker build -t spark-iceberg:3.5.1 spark-jobs/
14 docker tag spark-iceberg:3.5.1 ${ECR_REPO}:3.5.1
15 docker push ${ECR_REPO}:3.5.1

```

Listing 5.4: Build and push the Spark image

5.1.3 Bronze Layer Iceberg Writer Job

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import from_json, col, current_timestamp
3 from pyspark.sql.types import StructType, StringType
4 import argparse
5
6 def create_spark_session(app_name: str) -> SparkSession:
7     return (SparkSession.builder
8             .appName(app_name)
9             ## Iceberg catalog configuration
10            .config("spark.sql.extensions",
11                   "org.apache.iceberg.spark.extensions.
12                     IcebergSparkSessionExtensions")
13            .config("spark.sql.catalog.lakehouse",
14                   "org.apache.iceberg.spark.SparkCatalog")
15            .config("spark.sql.catalog.lakehouse.type", "hive")
16            .config("spark.sql.catalog.lakehouse.uri",
17                   "thrift://hive-metastore.platform-metastore.svc.cluster.
18                     local:9083")
19            ## S3A configuration for MinIO
20            .config("spark.hadoop.fs.s3a.endpoint",
21                   "http://minio.platform-storage.svc.cluster.local:9000")
22            .config("spark.hadoop.fs.s3a.path.style.access", "true")
23            .config("spark.hadoop.fs.s3a.impl",
24                   "org.apache.hadoop.fs.s3a.S3AFileSystem")
25            .config("spark.hadoop.fs.s3a.aws.credentials.provider",
26                   "org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
27            .getOrCreate())
28
29 def main():
30     parser = argparse.ArgumentParser()
31     parser.add_argument("--kafka-brokers", default="platform-kafka-
32       bootstrap.platform-ingestion.svc.cluster.local:9092")
33     parser.add_argument("--kafka-topic", required=True)
34     parser.add_argument("--target-table", required=True) # e.g.
35       lakehouse.bronze.agency_budget
36     parser.add_argument("--checkpoint-path", required=True) # s3a://
37       lakehouse/checkpoints/...
38     args = parser.parse_args()
39
40     spark = create_spark_session(f"bronze-writer-{args.kafka_topic}")
41
42     ## Create Iceberg table if not exists (schema-on-write)
43     spark.sql(f"""

```

```

39     CREATE TABLE IF NOT EXISTS {args.target_table} (
40         kafka_offset      BIGINT,
41         kafka_partition INT,
42         ingested_at       TIMESTAMP,
43         payload           STRING      -- raw JSON CDC event
44     )
45     USING iceberg
46     PARTITIONED BY (days(ingested_at))
47     TBLPROPERTIES (
48         'write.format.default'          = 'parquet',
49         'write.parquet.compression-codec' = 'zstd',
50         'write.metadata.compression-codec' = 'gzip',
51         'history.expire.max-snapshot-age-ms' = '604800000'
52     )
53     """
54
55     ## Structured Streaming: read from Kafka, write to Iceberg
56     df = (spark.readStream
57         .format("kafka")
58         .option("kafka.bootstrap.servers", args.kafka_brokers)
59         .option("subscribe", args.kafka_topic)
60         .option("startingOffsets", "earliest")
61         .option("failOnDataLoss", "false")
62         .load()
63         .selectExpr(
64             "CAST(offset AS BIGINT)      AS kafka_offset",
65             "partition                  AS kafka_partition",
66             "current_timestamp()         AS ingested_at",
67             "CAST(value AS STRING)       AS payload"
68         ))
69
70     query = (df.writeStream
71         .format("iceberg")
72         .outputMode("append")
73         .option("checkpointLocation", args.checkpoint_path)
74         .toTable(args.target_table)
75         .trigger(processingTime="30 seconds"))
76
77     query.awaitTermination()
78
79 if __name__ == "__main__":
80     main()

```

Listing 5.5: *spark-jobs/bronze_writer.py – Kafka to Bronze Iceberg*

```

1 apiVersion: sparkoperator.k8s.io/v1beta2
2 kind: SparkApplication
3 metadata:
4   name: bronze-agency-budget-writer
5   namespace: platform-compute
6 spec:
7   type: Python
8   pythonVersion: "3"

```



```
9   mode: cluster
10   image: "123456789.dkr.ecr.eu-west-1.amazonaws.com/spark-iceberg:3.5.1"
11   imagePullPolicy: Always
12   mainApplicationFile: "s3a://spark-jobs/bronze_writer.py"
13   arguments:
14     - "--kafka-topic=bronze.pg.public.agency_budget"
15     - "--target-table=lakehouse.bronze.agency_budget"
16     - "--checkpoint-path=s3a://lakehouse/checkpoints/bronze/agency_budget/"
17
18   sparkConf:
19     ## MinIO credentials (from Kubernetes secret)
20     "spark.hadoop.fs.s3a.access.key": "hive-sa"
21     "spark.hadoop.fs.s3a.secret.key": "SECRET_INJECTED_BY_INIT_CONTAINER"
22
23   envFrom:
24     - secretRef:
25         name: minio-spark-credentials      # Injects MINIO_ACCESS_KEY /
26                                           MINIO_SECRET_KEY
27
28   driver:
29     cores: 1
30     coreLimit: "1200m"
31     memory: "2g"
32     serviceAccount: spark-sa
33     nodeSelector:
34       node-role: compute
35
36   executor:
37     cores: 2
38     instances: 2
39     memory: "4g"
40     nodeSelector:
41       node-role: compute
42
43   restartPolicy:
44     type: OnFailure
45     onFailureRetries: 3
46     onFailureRetryInterval: 30
47     onSubmissionFailureRetries: 5
```

Listing 5.6: *k8s/spark/bronze-writer-job.yaml – SparkApplication CRD*

5.2 dbt Project Setup

```
1  pip3 install dbt-spark[PyHive]==1.7.3
2
3  ## Initialize dbt project
4  cd dbt/
5  dbt init data_platform
6  ## Choose: spark adapter
7
8  ## dbt/profiles.yml (stored in ~/.dbt/profiles.yml, NOT in git)
```

```

9 data_platform:
10   target: dev
11   outputs:
12     dev:
13       type: spark
14       method: thrift
15       host: hive-metastore.platform-metastore.svc.cluster.local
16       port: 9083
17       user: dbt
18       schema: silver      ## default target schema
19       connect_retries: 5
20       connect_timeout: 60

```

Listing 5.7: Set up dbt with Spark adapter

```

1 {{
2   config(
3     materialized = 'incremental',
4     incremental_strategy = 'merge',
5     unique_key    = ['code_agence', 'moisannee'],
6     file_format   = 'iceberg',
7     partition_by  = {'field': 'moisannee', 'data_type': 'string'},
8     table_properties = {
9       'write.format.default': 'parquet',
10      'write.parquet.compression-codec': 'zstd'
11    }
12  )
13 }}
14
15 WITH raw AS (
16   SELECT
17     JSON_VALUE(payload, '$.code_agence') AS code_agence,
18     JSON_VALUE(payload, '$.lib_agence') AS lib_agence,
19     JSON_VALUE(payload, '$.moisannee') AS moisannee,
20     CAST(JSON_VALUE(payload, '$.montant_calcule') AS DECIMAL(18, 2)) AS
      montant_calcule,
21     ingested_at
22 FROM {{ source('bronze', 'agency_budget') }}
23
24 {% if is_incremental() %}
25   WHERE ingested_at > (SELECT MAX(ingested_at) FROM {{ this }})
26 {% endif %}
27 ),
28
29 deduplicated AS (
30   SELECT *,
31     ROW_NUMBER() OVER (
32       PARTITION BY code_agence, moisannee
33       ORDER BY ingested_at DESC
34     ) AS rn
35 FROM raw
36 WHERE code_agence IS NOT NULL
37       AND moisannee IS NOT NULL

```

```

38     AND montant_calcule > 0
39 )
40
41 SELECT
42     code_agence ,
43     lib_agence ,
44     moisannee ,
45     montant_calcule ,
46     ingested_at ,
47     CURRENT_TIMESTAMP() AS transformed_at
48 FROM deduplicated
49 WHERE rn = 1

```

Listing 5.8: *dbt/models/silver/stg_agency_budget.sql*

```

1  {{
2      config(
3          materialized = 'table',
4          file_format  = 'iceberg',
5          partition_by = {'field': 'year_month', 'data_type': 'string'},
6      )
7  }}
8
9  SELECT
10     SUBSTR(moisannee, 1, 7)           AS year_month,
11     code_agence ,
12     lib_agence ,
13     SUM(montant_calcule)              AS total_budget ,
14     COUNT(*)                         AS record_count ,
15     MAX(transformed_at)              AS last_updated
16 FROM {{ ref('stg_agency_budget') }}
17 GROUP BY 1, 2, 3

```

Listing 5.9: *dbt/models/gold/agg_monthly_budget.sql*

5.3 Apache Airflow

```

1  executor: KubernetesExecutor
2
3  ## Git-sync for DAGs (DAGs are version-controlled, not baked into image)
4  dags:
5      gitSync:
6          enabled: true
7          repo: "https://github.com/your-org/data-platform"
8          branch: main
9          subPath: "dags"
10         period: 60s # Sync every 60 seconds
11
12  ## Disable built-in PostgreSQL -- use our CloudNativePG cluster
13  postgresql:
14      enabled: false
15  data:
16      metadataConnection:

```

```
17   user: airflow
18   pass: "" # Set via --set or external secret
19   host: pg-airflow-rw.platform-orchestr.svc.cluster.local
20   port: 5432
21   db: airflow
22
23 ## Airflow configuration
24 config:
25   core:
26     load_examples: "False"
27     max_active_tasks_per_dag: "16"
28   kubernetes_executor:
29     namespace: platform-compute # Spawn task pods here
30     worker_container_repository: apache/airflow
31     worker_container_tag: "2.9.1"
32
33 ## Fernet key for secret encryption (generate once, never change)
34 fernetKey: "" ## Set via: python -c "from cryptography.fernet import
    Fernet; print(Fernet.generate_key().decode())"
35
36 webserver:
37   replicas: 1
38   resources:
39     requests: { cpu: "500m", memory: "1Gi" }
40
41 scheduler:
42   replicas: 1
43   resources:
44     requests: { cpu: "500m", memory: "1Gi" }
45
46 nodeSelector:
47   node-role: compute
```

Listing 5.10: *helm/airflow/values.yaml (key sections)*

```
1 helm repo add apache-airflow https://airflow.apache.org
2 helm repo update
3
4 ## Create Airflow PostgreSQL cluster first
5 kubectl apply -f k8s/databases/postgres-airflow.yaml
6
7 FERNET_KEY=$(python3 -c \
8   "from cryptography.fernet import Fernet; print(Fernet.generate_key().
9     decode())")
10
11 helm install airflow apache-airflow/airflow \
12   --namespace platform-orchestr \
13   --version 1.13.1 \
14   --values helm/airflow/values.yaml \
15   --set fernetKey="${FERNET_KEY}" \
16   --set data.metadataConnection.pass="$(kubectl get secret \
17     pg-airflow-user-secret -n platform-orchestr \
18     -o jsonpath='{.data.password}' | base64 -d)"
```

```
18
19 kubectl rollout status deploy/airflow-webserver -n platform-orchestr
```

Listing 5.11: Deploy Airflow

```
1 from datetime import datetime, timedelta
2 from airflow import DAG
3 from airflow.providers.apache.spark.operators.spark_submit import
  SparkSubmitOperator
4 from airflow.operators.bash import BashOperator
5
6 default_args = {
7     "owner": "data-engineering",
8     "retries": 2,
9     "retry_delay": timedelta(minutes=5),
10    "execution_timeout": timedelta(hours=2),
11 }
12
13 with DAG(
14     dag_id="bronze_to_silver_agency_budget",
15     default_args=default_args,
16     schedule_interval="0 9 * * *",      # Run at 9 AM daily (after scale-up at
        8AM)
17     start_date=datetime(2025, 1, 1),
18     catchup=False,
19     tags=["silver", "budget", "daily"],
20 ) as dag:
21
22     ## Step 1: Run dbt Silver models
23     dbt_silver = BashOperator(
24         task_id="dbt_run_silver",
25         bash_command="""
26             dbt run \
27                 --project-dir /opt/dbt/data_platform \
28                 --profiles-dir /opt/dbt \
29                 --models silver.stg_agency_budget \
30                 --target dev
31         """,
32     )
33
34     ## Step 2: Run dbt Gold models
35     dbt_gold = BashOperator(
36         task_id="dbt_run_gold",
37         bash_command="""
38             dbt run \
39                 --project-dir /opt/dbt/data_platform \
40                 --profiles-dir /opt/dbt \
41                 --models gold.agg_monthly_budget \
42                 --target dev
43         """,
44     )
45
46     ## Step 3: Run dbt tests
```

```
47     dbt_test = BashOperator(  
48         task_id="dbt_test",  
49         bash_command="""  
50             dbt test \  
51                 --project-dir /opt/dbt/data_platform \  
52                 --profiles-dir /opt/dbt \  
53                 --models silver gold  
54             """,  
55     )  
56  
57     dbt_silver >> dbt_gold >> dbt_test
```

Listing 5.12: *days/bronze_to_silver_pipeline.py*

✓ Checkpoint 4: Phase 4 Complete

- Spark Operator with custom Iceberg-enabled Docker image
- Bronze writer SparkApplication reading from Kafka
- dbt project with Silver (incremental) and Gold (table) models
- Airflow with KubernetesExecutor and Git-synced DAGs
- End-to-end pipeline: Kafka → Bronze → Silver → Gold

6 | Phase 5 – Serving Layer: Dremio & Power BI

6.1 Dremio Deployment

```
1  ## Using the official Dremio Helm chart
2  image:
3    repository: dremio/dremio-oss
4    tag: "24.1.0"
5    pullPolicy: IfNotPresent
6
7  ## Coordinator: handles query planning and UI
8  coordinator:
9    cpu: 4
10   memory: 16384          # MB
11   count: 1               # 1 coordinator for MVP
12   nodeSelector:
13     node-role: compute
14
15   ## Dremio internal storage (metadata, catalog)
16   volumeClaimTemplate:
17     storageClass: gp3
18     resources:
19       requests:
20         storage: 100Gi
21
22   ## Executors: handle actual query execution
23   executor:
24     cpu: 4
25     memory: 16384
26     count: 2             # 2 executors for MVP
27     nodeSelector:
28       node-role: compute
29     volumeClaimTemplate:
30       storageClass: gp3
31       resources:
32         requests:
33           storage: 200Gi
34
35   ## Zookeeper for distributed coordination
36   zookeeper:
37     count: 1              # MVP: single ZK; prod: 3
38     cpu: 0.5
39     memory: 1024
```

```

40   nodeSelector:
41     node-role: compute
42
43   ## Service configuration
44   service:
45     type: ClusterIP
46     internalLoadBalancer: false
47
48   ## Dremio configuration
49   dremio_config:
50     ## Arrow Flight service (used by Power BI)
51     services.flight.enabled: "true"
52     services.flight.port: "32010"

```

Listing 6.1: *helm/dremio/values.yaml*

```

1 helm repo add dremio https://charts.dremio.com
2 helm repo update
3
4 helm install dremio dremio/dremio \
5   --namespace platform-serving \
6   --version 24.1.0 \
7   --values helm/dremio/values.yaml
8
9 ## Wait for coordinator to be ready (5-8 minutes)
10 kubectl rollout status statefulset/dremio-coordinator -n platform-serving
11
12 ## Port-forward for initial setup
13 kubectl port-forward svc/dremio-client -n platform-serving 9047:9047 &
14 ## Access UI: http://localhost:9047
15 ## Set admin username/password on first launch

```

Listing 6.2: *Install Dremio*

6.1.1 Connect Dremio to Hive Metastore

```

1 ## Get Dremio auth token
2 DREMIO_TOKEN=$(curl -s -X POST http://localhost:9047/apiv2/login \
3   -H "Content-Type: application/json" \
4   -d '{"userName":"admin","password":"your-dremio-password"}' \
5   | python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")
6
7 ## Add Hive Metastore source
8 curl -X POST http://localhost:9047/apiv2/catalog \
9   -H "Content-Type: application/json" \
10  -H "Authorization: _dremio${DREMIO_TOKEN}" \
11  -d '{
12    "entityType": "source",
13    "name": "lakehouse",
14    "type": "HIVE",
15    "config": {
16      "hostname": "hive-metastore.platform-metastore.svc.cluster.local",
17      "port": 9083,

```



```

18     "enableAsync": true,
19     "authType": "STORAGE",
20     "propertyList": [
21       {"name": "fs.s3a.endpoint",
22        "value": "http://minio.platform-storage.svc.cluster.local:9000"},
23       {"name": "fs.s3a.path.style.access", "value": "true"},
24       {"name": "fs.s3a.impl",
25        "value": "org.apache.hadoop.fs.s3a.S3AFileSystem"},
26       {"name": "fs.s3a.access.key", "value": "dremio-sa"},
27       {"name": "fs.s3a.secret.key", "value": "DREMIO_MINIO_SECRET"}
28     ]
29   }
30 },
31
32 ## Verify: browse to http://localhost:9047 -> Sources -> lakehouse
33 ## You should see: bronze, silver, gold schemas with Iceberg tables

```

Listing 6.3: Configure Dremio source via REST API

6.2 Power BI Connection

```

1 ## 1. Download Dremio ODBC driver
2 ##   https://www.dremio.com/drivers/odbc/
3
4 ## 2. Configure ODBC Data Source
5 ##   Control Panel -> ODBC Data Sources (64-bit) -> Add
6 ##   Driver: Dremio Connector
7 ##   Host:   dremio.dataplatform.example.com   (Cloudflare tunnel FQDN)
8 ##   Port:   32010                             (Arrow Flight port)
9 ##   Auth:   Username / Password (Dremio PAT)
10
11 ## 3. In Power BI Desktop
12 ##   Get Data -> ODBC -> Select the Dremio DSN
13 ##   Navigator -> lakehouse -> gold -> agg_monthly_budget
14 ##   Load -> Transform -> Publish to Power BI Service

```

Listing 6.4: Power BI connection steps (on Windows machine)

7 | Phase 6 – Security: Cloudflare Zero Trust

7.1 Cloudflare Operator

```
1 ## 1. Cloudflare Dashboard -> My Profile -> API Tokens -> Create Token
2 ##   Template: "Edit Cloudflare Workers" + add:
3 ##   Zone:DNS>Edit, Account:Cloudflare Tunnel>Edit
4
5 ## 2. Create Kubernetes secret with the API token
6 kubectl create secret generic cloudflare-credentials \
7   --namespace platform-security \
8   --from-literal=CLOUDFLARE_API_TOKEN="<your-cf-api-token>" \
9   --from-literal=CLOUDFLARE_ACCOUNT_ID="<your-cf-account-id>"
10
11 ## 3. Install Cloudflare Tunnel Operator
12 helm repo add cloudflare https://cloudflare.github.io/helm-charts
13 helm repo update
14
15 helm install cloudflare-operator cloudflare/cloudflare-operator \
16   --namespace platform-security \
17   --version 0.2.1 \
18   --set config.apiToken.secretName=cloudflare-credentials \
19   --set config.apiToken.secretKey=CLOUDFLARE_API_TOKEN \
20   --set config.accountId.secretName=cloudflare-credentials \
21   --set config.accountId.secretKey=CLOUDFLARE_ACCOUNT_ID
22
23 kubectl rollout status deploy/cloudflare-operator -n platform-security
```

Listing 7.1: Create Cloudflare API token and deploy operator

```
1 apiVersion: networking.cfargotunnel.com/v1alpha1
2 kind: ClusterTunnel
3 metadata:
4   name: dataplatform-tunnel
5   namespace: platform-security
6 spec:
7   newTunnel:
8     name: dataplatform-mvp
9   cloudflare:
10     domain: dataplatform.example.com    ## Your Cloudflare-managed domain
11     email: admin@example.com
12     accountId:
13       secretRef:
14         name: cloudflare-credentials
15         key: CLOUDFLARE_ACCOUNT_ID
16     apiToken:
```

```
17     secretRef:
18       name: cloudflare-credentials
19       key: CLOUDFLARE_API_TOKEN
```

Listing 7.2: *k8s/cloudflare/tunnel.yaml – Create the ClusterTunnel*

```
1 ---
2 ## Airflow
3 apiVersion: networking.cfargotunnel.com/v1alpha1
4 kind: TunnelBinding
5 metadata:
6   name: airflow
7   namespace: platform-orchestr
8 spec:
9   subjects:
10    - name: airflow-webserver
11      spec:
12        fqdn: airflow.dataplatform.example.com
13        protocol: http
14        servicePort: 8080
15      tunnelRef:
16        kind: ClusterTunnel
17        name: dataplatform-tunnel
18 ---
19 ## Dremio UI
20 apiVersion: networking.cfargotunnel.com/v1alpha1
21 kind: TunnelBinding
22 metadata:
23   name: dremio
24   namespace: platform-serving
25 spec:
26   subjects:
27    - name: dremio-client
28      spec:
29        fqdn: dremio.dataplatform.example.com
30        protocol: http
31        servicePort: 9047
32      tunnelRef:
33        kind: ClusterTunnel
34        name: dataplatform-tunnel
35 ---
36 ## MinIO Console
37 apiVersion: networking.cfargotunnel.com/v1alpha1
38 kind: TunnelBinding
39 metadata:
40   name: minio-console
41   namespace: platform-storage
42 spec:
43   subjects:
44    - name: lakehouse-console
45      spec:
46        fqdn: minio.dataplatform.example.com
47        protocol: https
```

```
48     servicePort: 9090
49     noTLSVerify: true           # MinIO uses self-signed cert internally
50   tunnelRef:
51     kind: ClusterTunnel
52     name: dataplatform-tunnel
53 ---
54 ## Grafana
55 apiVersion: networking.cfargotunnel.com/v1alpha1
56 kind: TunnelBinding
57 metadata:
58   name: grafana
59   namespace: platform-monitoring
60 spec:
61   subjects:
62     - name: monitoring-grafana
63     spec:
64       fqdn: grafana.dataplatform.example.com
65       protocol: http
66       servicePort: 80
67   tunnelRef:
68     kind: ClusterTunnel
69     name: dataplatform-tunnel
```

Listing 7.3: *k8s/cloudflare/tunnel-bindings.yaml – Expose all services*

```
1 kubectl apply -f k8s/cloudflare/tunnel.yaml
2 kubectl apply -f k8s/cloudflare/tunnel-bindings.yaml
3
4 ## Verify tunnel is connected
5 kubectl get clustertunnel dataplatform-tunnel -n platform-security
6 ## Status should show: TunnelHealthy: True
7
8 ## Verify DNS records are created in Cloudflare (takes 1-2 minutes)
9 ## Check Cloudflare dashboard -> DNS -> airflow.dataplatform.example.com
```

Listing 7.4: *Apply all tunnel bindings*

8 | Phase 7 – Observability: Prometheus, Grafana & OpenObserve

8.1 kube-prometheus-stack

```
1  ## Grafana configuration
2  grafana:
3    enabled: true
4    adminPassword: ""      ## Set via --set grafana.adminPassword=...
5    persistence:
6      enabled: true
7      storageClassName: gp3
8      size: 10Gi
9    nodeSelector:
10     node-role: compute
11    ## Pre-load dashboards for our stack
12    dashboardProviders:
13     dashboardproviders.yaml:
14       apiVersion: 1
15       providers:
16         - name: 'default'
17           folder: 'Data Platform'
18           type: file
19           options:
20             path: /var/lib/grafana/dashboards/default
21    ## Import community dashboards by ID
22    dashboardsConfigMaps:
23     - configMapName: grafana-dashboards
24       path: /var/lib/grafana/dashboards/default
25
26  ## Prometheus configuration
27  prometheus:
28    prometheusSpec:
29     ## Watch ALL namespaces for ServiceMonitors
30     serviceMonitorSelectorNilUsesHelmValues: false
31     podMonitorSelectorNilUsesHelmValues: false
32     retention: 30d
33     retentionSize: "50GB"
34     storageSpec:
35       volumeClaimTemplate:
36         spec:
37           storageClassName: gp3
38           resources:
39             requests:
```

```

40         storage: 100Gi
41     nodeSelector:
42         node-role: compute
43
44     ## AlertManager
45     alertmanager:
46         alertmanagerSpec:
47             storage:
48                 volumeClaimTemplate:
49                     spec:
50                         storageClassName: gp3
51                         resources:
52                             requests:
53                                 storage: 10Gi
54             nodeSelector:
55                 node-role: compute
56         config:
57             global:
58                 slack_api_url: 'https://hooks.slack.com/services/...'
59             route:
60                 group_by: ['namespace', 'alertname']
61                 receiver: 'slack-notifications'
62             receivers:
63                 - name: 'slack-notifications'
64                   slack_configs:
65                       - channel: '#data-platform-alerts'
66                         text: "{{ range .Alerts }}{{ .Annotations.description }}\n{{ end }}"
67
68     ## Node exporter on ALL nodes
69     nodeExporter:
70         enabled: true
71
72     ## kube-state-metrics for K8s object metrics
73     kubeStateMetrics:
74         enabled: true

```

Listing 8.1: *helm/monitoring/values.yaml*

```

1 helm repo add prometheus-community \
2     https://prometheus-community.github.io/helm-charts
3 helm repo update
4
5 GRAFANA_PASS=$(openssl rand -base64 16)
6
7 helm install monitoring prometheus-community/kube-prometheus-stack \
8     --namespace platform-monitoring \
9     --version 58.3.1 \
10    --values helm/monitoring/values.yaml \
11    --set grafana.adminPassword="${GRAFANA_PASS}"
12
13 echo "Grafana password: $GRAFANA_PASS"
14

```

```

15 kubectl rollout status deploy/monitoring-grafana -n platform-monitoring
16 kubectl rollout status statefulset/prometheus-monitoring-kube-prometheus -n
    platform-monitoring

```

Listing 8.2: Install monitoring stack

8.2 Fluent Bit – Log Shipping

```

1  ## Fluent Bit DaemonSet configuration
2  daemonSetVolumes:
3    - name: varlog
4      hostPath:
5        path: /var/log
6    - name: varlibdockercontainers
7      hostPath:
8        path: /var/lib/docker/containers
9
10 daemonSetVolumeMounts:
11   - name: varlog
12     mountPath: /var/log
13   - name: varlibdockercontainers
14     mountPath: /var/lib/docker/containers
15     readOnly: true
16
17 config:
18   service: |
19     [SERVICE]
20       Flush      5
21       Daemon     Off
22       Log_Level  info
23       HTTP_Server On
24       HTTP_Listen 0.0.0.0
25       HTTP_Port  2020
26
27   inputs: |
28     [INPUT]
29       Name          tail
30       Path           /var/log/containers/*.log
31       multiline.parser docker, cri
32       Tag            kube.*
33       Mem_Buf_Limit  50MB
34       Skip_Long_Lines Off
35       Refresh_Interval 10
36
37   filters: |
38     [FILTER]
39       Name          kubernetes
40       Match          kube.*
41       Kube_URL       https://kubernetes.default.svc:443
42       Kube_CA_File   /var/run/secrets/kubernetes.io/serviceaccount/ca
43                       .crt
44       Kube_Token_File /var/run/secrets/kubernetes.io/serviceaccount/

```

```

    token
44     Kube_Tag_Prefix      kube.var.log.containers.
45     Merge_Log            On
46     Merge_Log_Key        log_processed
47     K8S-Logging.Parser   On
48     K8S-Logging.Exclude  Off
49
50     ## Drop health check logs to reduce noise
51     [FILTER]
52         Name      grep
53         Match     kube.*
54         Exclude   log    .*health.*check.*
55
56     outputs: |
57         [OUTPUT]
58             Name      s3
59             Match     kube.*
60             bucket     logs
61             region     us-east-1
62             endpoint   http://minio.platform-storage.svc.
63                 cluster.local:9000
64             use_put_object    On
65             s3_key_format     /year=%Y/month=%m/day=%d/hour=%H/%H-%M
66                 -%S-$UUID.gz
67             total_file_size   10M
68             upload_timeout    5m
69             compression       gzip
70             log_key            log
71             json_date_key      timestamp
72             json_date_format   iso8601
73             ## MinIO credentials
74             store_dir          /tmp/fluent-bit-s3

```

Listing 8.3: *helm/logging/fluentbit-values.yaml*

```

1  ## Fluent Bit
2  helm repo add fluent https://fluent.github.io/helm-charts
3  helm repo update
4
5  ## Create MinIO credentials for Fluent Bit
6  kubectl create secret generic fluentbit-minio-credentials \
7      --namespace platform-logging \
8      --from-literal=AWS_ACCESS_KEY_ID=fluent-sa \
9      --from-literal=AWS_SECRET_ACCESS_KEY="<fluent-minio-secret>"
10
11 helm install fluent-bit fluent/fluent-bit \
12     --namespace platform-logging \
13     --version 0.46.7 \
14     --values helm/logging/fluentbit-values.yaml \
15     --set envFrom[0].secretRef.name=fluentbit-minio-credentials
16
17 ## OpenObserve
18 helm repo add openobserve https://charts.openobserve.ai

```



```

19 helm repo update
20
21 helm install openobserve openobserve/openobserve \
22   --namespace platform-logging \
23   --version 0.10.0 \
24   --set auth.rootUserEmail="admin@example.com" \
25   --set auth.rootUserPassword="$(openssl rand -base64 16)" \
26   --set storage.s3.bucket=logs \
27   --set storage.s3.endpoint="http://minio.platform-storage.svc.cluster.local
   :9000" \
28   --set storage.s3.accessKey=openobserve-sa \
29   --set storage.s3.secretKey="<openobserve-minio-secret>"

```

Listing 8.4: *Deploy Fluent Bit and OpenObserve*

✓ Checkpoint 7: Phase 6 & 7 Complete

- Cloudflare Tunnel exposing 5 services with Zero Trust protection
- All services accessible via HTTPS FQDNs (no open firewall ports)
- Prometheus scraping all namespaces; 30-day retention
- Grafana with platform dashboards; Slack alerting configured
- Fluent Bit shipping logs to MinIO; OpenObserve ingesting from S3

9 | Phase 8 – CI/CD: GitHub Actions Pipeline

9.1 OIDC Trust Between GitHub and AWS

```
1  ## Create OIDC provider for GitHub Actions
2  resource "aws_iam_openid_connect_provider" "github" {
3      url          = "https://token.actions.githubusercontent.com"
4      client_id_list = ["sts.amazonaws.com"]
5      thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
6  }
7
8  ## IAM role that GitHub Actions will assume
9  resource "aws_iam_role" "github_actions" {
10     name = "${var.cluster_name}-github-actions"
11
12     assume_role_policy = jsonencode({
13         Version = "2012-10-17"
14         Statement = [{
15             Effect      = "Allow"
16             Principal = {
17                 Federated = aws_iam_openid_connect_provider.github.arn
18             }
19             Action = "sts:AssumeRoleWithWebIdentity"
20             Condition = {
21                 StringEquals = {
22                     "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
23                 }
24                 StringLike = {
25                     ## Only allow from your specific repo
26                     "token.actions.githubusercontent.com:sub" = [
27                         "repo:your-org/data-platform:ref:refs/heads/main",
28                         "repo:your-org/data-platform:pull_request"
29                     ]
30                 }
31             }
32         }]
33     })
34 }
35
36 ## Attach Terraform permissions policy
37 resource "aws_iam_role_policy_attachment" "github_actions_terraform" {
38     role          = aws_iam_role.github_actions.name
39     policy_arn    = aws_iam_policy.terraform_policy.arn
40 }
41
42 output "github_actions_role_arn" {
```

```

43     value = aws_iam_role.github_actions.arn
44 }

```

Listing 9.1: *terraform/modules/iam/github-oidc.tf*

9.2 GitHub Actions Workflows

```

1  name: "Terraform Infrastructure"
2
3  on:
4    push:
5      branches: [main]
6      paths: ["terraform/**"]
7    pull_request:
8      branches: [main]
9      paths: ["terraform/**"]
10
11  permissions:
12    id-token: write      # Required for OIDC
13    contents: read
14    pull-requests: write
15
16  env:
17    TF_VERSION: "1.7.5"
18    WORKING_DIR: "./terraform/environments/dev"
19    AWS_REGION: "eu-west-1"
20
21  jobs:
22    ## Lint & Validate
23    validate:
24      name: "Lint & Validate"
25      runs-on: ubuntu-latest
26      steps:
27        - uses: actions/checkout@v4
28
29        - uses: hashicorp/setup-terraform@v3
30          with:
31            terraform_version: "${ env.TF_VERSION }"
32
33        - name: Configure AWS (OIDC)
34          uses: aws-actions/configure-aws-credentials@v4
35          with:
36            role-to-assume: "${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }"
37            aws-region: "${ env.AWS_REGION }"
38
39        - name: Terraform Format Check
40          run: terraform fmt -check -recursive
41          working-directory: terraform/
42
43        - name: Terraform Init
44          run: terraform init -backend=false
45          working-directory: "${ env.WORKING_DIR }"

```

```

46     env:
47         AWS_ACCESS_KEY_ID:      ${ secrets.CF_R2_ACCESS_KEY }}
48         AWS_SECRET_ACCESS_KEY: ${ secrets.CF_R2_SECRET_KEY }}
49
50     - name: Terraform Validate
51       run: terraform validate
52       working-directory: ${ env.WORKING_DIR }}
53
54     - name: tflint
55       uses: terraform-linters/setup-tflint@v4
56       with:
57         tflint_version: v0.50.3
58
59     - run: |
60         tflint --init
61         tflint --recursive
62       working-directory: terraform/
63
64   ## Plan
65   plan:
66     name: "Terraform Plan"
67     runs-on: ubuntu-latest
68     needs: validate
69     outputs:
70       exitcode: ${ steps.plan.outputs.exitcode }}
71
72   steps:
73     - uses: actions/checkout@v4
74     - uses: hashicorp/setup-terraform@v3
75       with:
76         terraform_version: ${ env.TF_VERSION }}
77
78     - name: Configure AWS (OIDC)
79       uses: aws-actions/configure-aws-credentials@v4
80       with:
81         role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }}
82         aws-region: ${ env.AWS_REGION }}
83
84     - name: Terraform Init
85       run: terraform init
86       working-directory: ${ env.WORKING_DIR }}
87     env:
88         AWS_ACCESS_KEY_ID:      ${ secrets.CF_R2_ACCESS_KEY }}
89         AWS_SECRET_ACCESS_KEY: ${ secrets.CF_R2_SECRET_KEY }}
90
91     - name: Terraform Plan
92       id: plan
93       run: |
94         terraform plan \
95           -out=tfplan \
96           -detailed-exitcode \
97           -no-color \

```

```

98         2>&1 | tee plan_output.txt
99         echo "exitcode=$?" >> $GITHUB_OUTPUT
100     working-directory: ${ env.WORKING_DIR }}
101     continue-on-error: true
102
103     ## Post plan output as PR comment
104     - name: Post Plan to PR
105       if: github.event_name == 'pull_request'
106       uses: actions/github-script@v7
107       with:
108         script: |
109             const fs = require('fs');
110             const plan = fs.readFileSync(`${ env.WORKING_DIR }}/plan_output
111               .txt`, 'utf8');
112             const truncated = plan.length > 60000 ? plan.slice(-60000) + '\n
113               ...(truncated)' : plan;
114             github.rest.issues.createComment({
115               issue_number: context.issue.number,
116               owner: context.repo.owner,
117               repo: context.repo.repo,
118               body: '## Terraform Plan\n``hcl\n' + truncated + '\n``',
119             });
120
121     - name: Upload Plan
122       uses: actions/upload-artifact@v4
123       with:
124         name: tfplan
125         path: ${ env.WORKING_DIR }}/tfplan
126
127     ## Apply
128     apply:
129       name: "Terraform Apply"
130       runs-on: ubuntu-latest
131       needs: plan
132       if: |
133         github.ref == 'refs/heads/main' &&
134         github.event_name == 'push' &&
135         needs.plan.outputs.exitcode == '2'      # 2 = changes pending
136       environment: production                    # Requires manual approval in
137       GitHub
138
139     steps:
140       - uses: actions/checkout@v4
141       - uses: hashicorp/setup-terraform@v3
142       with:
143         terraform_version: ${ env.TF_VERSION }}
144
145     - name: Configure AWS (OIDC)
146       uses: aws-actions/configure-aws-credentials@v4
147       with:
148         role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }}
149         aws-region: ${ env.AWS_REGION }}

```

```

147
148     - name: Terraform Init
149       run: terraform init
150       working-directory: ${ env.WORKING_DIR }}
151       env:
152         AWS_ACCESS_KEY_ID:      ${ secrets.CF_R2_ACCESS_KEY }}
153         AWS_SECRET_ACCESS_KEY:  ${ secrets.CF_R2_SECRET_KEY }}
154
155     - name: Download Plan
156       uses: actions/download-artifact@v4
157       with:
158         name: tfplan
159         path: ${ env.WORKING_DIR }}
160
161     - name: Terraform Apply
162       run: terraform apply -auto-approve tfplan
163       working-directory: ${ env.WORKING_DIR }}

```

Listing 9.2: `.github/workflows/terraform.yml`

```

1 name: "Helm Deploy"
2
3 on:
4   push:
5     branches: [main]
6     paths: ["helm/**", "k8s/**"]
7
8 jobs:
9   helm-deploy:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v4
13
14       - name: Configure AWS (OIDC)
15         uses: aws-actions/configure-aws-credentials@v4
16         with:
17           role-to-assume: ${ secrets.AWS_GITHUB_ACTIONS_ROLE_ARN }}
18           aws-region: eu-west-1
19
20       - name: Update kubeconfig
21         run: |
22           aws eks update-kubeconfig \
23             --region eu-west-1 \
24             --name dataplatform-dev
25
26       - name: Install Helm
27         uses: azure/setup-helm@v3
28         with:
29           version: 'v3.14.3'
30
31       - name: Deploy/Upgrade all charts
32         run: |
33           ## Update all Helm repos

```

```

34     helm repo add strimzi      https://strimzi.io/charts/
35     helm repo add cnpg        https://cloudnative-pg.github.io/charts
36     helm repo add spark-operator https://kubeflow.github.io/spark-
        operator
37     helm repo add apache-airflow https://airflow.apache.org
38     helm repo update
39
40     ## Apply Kubernetes manifests
41     kubectl apply -f k8s/namespaces/
42     kubectl apply -f k8s/rbac/
43     kubectl apply -f k8s/storage-classes/
44     kubectl apply -f k8s/network-policies/
45
46     ## Upgrade charts (idempotent)
47     helm upgrade --install spark-operator spark-operator/spark-
        operator \
48         --namespace platform-compute \
49         --values helm/spark-operator/values.yaml
50
51     helm upgrade --install airflow apache-airflow/airflow \
52         --namespace platform-orchestr \
53         --values helm/airflow/values.yaml \
54         --set fernetKey="${ secrets.AIRFLOW_FERNET_KEY }"
55
56 - name: Smoke Test
57   run: |
58     ## Verify all critical pods are running
59     kubectl wait pods \
60         --all \
61         --namespace platform-compute \
62         --for=condition=Ready \
63         --timeout=180s
64
65     kubectl wait pods \
66         --all \
67         --namespace platform-storage \
68         --for=condition=Ready \
69         --timeout=180s
70
71     echo "All pods are Ready -- deploy successful"

```

Listing 9.3: `.github/workflows/helm-deploy.yml`

9.3 GitHub Repository Secrets Setup

 Required GitHub Secrets

Navigate to your repo → Settings → Secrets and Variables → Actions, and add:

Secret Name	Value
<code>AWS_GITHUB_ACTIONS_ROLE_ARN</code>	ARN from Terraform IAM module output
<code>CF_R2_ACCESS_KEY</code>	Cloudflare R2 access key ID
<code>CF_R2_SECRET_KEY</code>	Cloudflare R2 secret access key
<code>AIRFLOW_FERNET_KEY</code>	Fernet key generated during Airflow setup
<code>GRAFANA_ADMIN_PASSWORD</code>	Grafana admin password
<code>MINIO_ROOT_PASSWORD</code>	MinIO root password

10 | Phase 9 – Cost Control: Lambda Node Scheduling

```
1  ## Package the Lambda function
2  data "archive_file" "scaling_lambda" {
3      type          = "zip"
4      source_dir    = "${path.module}/src"
5      output_path   = "${path.module}/lambda_scaling.zip"
6  }
7
8  resource "aws_lambda_function" "eks_scale_up" {
9      function_name    = "${var.cluster_name}-scale-up"
10     filename         = data.archive_file.scaling_lambda.output_path
11     source_code_hash  = data.archive_file.scaling_lambda.output_base64sha256
12     runtime          = "python3.12"
13     handler          = "scale.scale_up_handler"
14     role              = aws_iam_role.lambda_eks.arn
15     timeout          = 60
16
17     environment {
18         variables = {
19             CLUSTER_NAME      = var.cluster_name
20             NODEGROUP_NAME    = var.nodegroup_name
21             MIN_SIZE          = tostring(var.min_size_up)
22             DESIRED_SIZE      = tostring(var.desired_up)
23             MAX_SIZE          = "10"
24             AWS_REGION_NAME   = var.aws_region
25         }
26     }
27
28     tags = { Purpose = "eks-cost-control" }
29 }
30
31 resource "aws_lambda_function" "eks_scale_down" {
32     function_name    = "${var.cluster_name}-scale-down"
33     filename         = data.archive_file.scaling_lambda.output_path
34     source_code_hash  = data.archive_file.scaling_lambda.output_base64sha256
35     runtime          = "python3.12"
36     handler          = "scale.scale_down_handler"
37     role              = aws_iam_role.lambda_eks.arn
38     timeout          = 60
39
40     environment {
41         variables = {
```

```

42     CLUSTER_NAME      = var.cluster_name
43     NODEGROUP_NAME    = var.nodegroup_name
44     MIN_SIZE          = "0"
45     DESIRED_SIZE      = "0"
46     MAX_SIZE          = "0"
47     AWS_REGION_NAME   = var.aws_region
48   }
49 }
50 }
51
52 ## EventBridge (CloudWatch Events) schedules
53 resource "aws_cloudwatch_event_rule" "scale_up" {
54   name           = "${var.cluster_name}-scale-up"
55   description    = "Scale up EKS compute node group at 8AM"
56   schedule_expression = "cron(${var.scale_up_cron})"
57   state          = "ENABLED"
58 }
59
60 resource "aws_cloudwatch_event_rule" "scale_down" {
61   name           = "${var.cluster_name}-scale-down"
62   description    = "Scale down EKS compute node group at 6PM"
63   schedule_expression = "cron(${var.scale_down_cron})"
64   state          = "ENABLED"
65 }
66
67 resource "aws_cloudwatch_event_target" "scale_up" {
68   rule = aws_cloudwatch_event_rule.scale_up.name
69   arn  = aws_lambda_function.eks_scale_up.arn
70 }
71 resource "aws_cloudwatch_event_target" "scale_down" {
72   rule = aws_cloudwatch_event_rule.scale_down.name
73   arn  = aws_lambda_function.eks_scale_down.arn
74 }
75
76 resource "aws_lambda_permission" "allow_eventbridge_up" {
77   statement_id = "AllowEventBridgeScaleUp"
78   action       = "lambda:InvokeFunction"
79   function_name = aws_lambda_function.eks_scale_up.function_name
80   principal     = "events.amazonaws.com"
81   source_arn    = aws_cloudwatch_event_rule.scale_up.arn
82 }
83 resource "aws_lambda_permission" "allow_eventbridge_down" {
84   statement_id = "AllowEventBridgeScaleDown"
85   action       = "lambda:InvokeFunction"
86   function_name = aws_lambda_function.eks_scale_down.function_name
87   principal     = "events.amazonaws.com"
88   source_arn    = aws_cloudwatch_event_rule.scale_down.arn
89 }

```

Listing 10.1: *terraform/modules/lambda-scaling/main.tf*

```

1 import boto3, os, logging
2

```

```
3 logger = logging.getLogger()
4 logger.setLevel(logging.INFO)
5
6 CLUSTER      = os.environ["CLUSTER_NAME"]
7 NODEGROUP    = os.environ["NODEGROUP_NAME"]
8 MIN_SIZE     = int(os.environ["MIN_SIZE"])
9 DESIRED      = int(os.environ["DESIRED_SIZE"])
10 MAX_SIZE     = int(os.environ["MAX_SIZE"])
11 REGION       = os.environ["AWS_REGION_NAME"]
12
13 def _scale(min_size: int, desired: int, max_size: int) -> dict:
14     eks = boto3.client("eks", region_name=REGION)
15     response = eks.update_nodegroup_config(
16         clusterName=CLUSTER,
17         nodegroupName=NODEGROUP,
18         scalingConfig={
19             "minSize": min_size,
20             "desiredSize": desired,
21             "maxSize": max_size,
22         }
23     )
24     logger.info("Scaling response: %s", response["update"]["status"])
25     return {
26         "statusCode": 200,
27         "cluster": CLUSTER,
28         "nodegroup": NODEGROUP,
29         "minSize": min_size,
30         "desired": desired,
31     }
32
33 def scale_up_handler(event, context):
34     logger.info("SCALE UP: %s/%s -> desired=%d", CLUSTER, NODEGROUP, DESIRED)
35     return _scale(MIN_SIZE, DESIRED, MAX_SIZE)
36
37 def scale_down_handler(event, context):
38     logger.info("SCALE DOWN: %s/%s -> desired=0", CLUSTER, NODEGROUP)
39     return _scale(0, 0, 0)
```

Listing 10.2: *terraform/modules/lambda-scaling/src/scale.py*

11 | Phase 10 – ArgoCD GitOps (Optional)

11.1 Install ArgoCD

```
1 helm repo add argo https://argoproj.github.io/argo-helm
2 helm repo update
3
4 helm install argocd argo/argo-cd \
5   --namespace argocd \
6   --create-namespace \
7   --version 6.7.18 \
8   --set server.service.type=ClusterIP \
9   --set configs.params."server\.\insecure"=true \
10  --set server.nodeSelector."node-role"=system \
11  --set server.tolerations[0].key=node-role \
12  --set server.tolerations[0].value=system \
13  --set server.tolerations[0].effect=NoSchedule
14
15 kubectl rollout status deploy/argocd-server -n argocd
16
17 ## Get initial admin password
18 ARGOCD_PASS=$(kubectl -n argocd get secret argocd-initial-admin-secret \
19   -o jsonpath='{.data.password}' | base64 -d)
20 echo "ArgoCD admin password: $ARGOCD_PASS"
21
22 ## Login via CLI
23 argocd login localhost:8080 \
24   --username admin \
25   --password "${ARGOCD_PASS}" \
26   --insecure &
27
28 ## Connect your GitHub repo
29 argocd repo add https://github.com/your-org/data-platform \
30   --username your-github-user \
31   --password "${GITHUB_TOKEN}"
```

Listing 11.1: Install ArgoCD

```
1 ## The "App of Apps" pattern: one root Application manages all others
2 apiVersion: argoproj.io/v1alpha1
3 kind: Application
4 metadata:
5   name: data-platform-root
6   namespace: argocd
7   finalizers:
8     - resources-finalizer.argocd.argoproj.io
```

```

9 spec:
10   project: default
11   source:
12     repoURL: https://github.com/your-org/data-platform
13     targetRevision: HEAD
14     path: k8s/argocd/apps          ## Directory containing child
                                   Application CRDs
15   destination:
16     server: https://kubernetes.default.svc
17     namespace: argocd
18   syncPolicy:
19     automated:
20       prune: true
21       selfHeal: true              ## Auto-fix cluster drift
22   syncOptions:
23     - CreateNamespace=true
24     - PrunePropagationPolicy=foreground
25     - RespectIgnoreDifferences=true

```

Listing 11.2: *k8s/argocd/app-of-apps.yaml* – App of Apps pattern

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: minio-operator
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/your-org/data-platform
10    targetRevision: HEAD
11    path: helm/minio-operator
12    helm:
13      valueFiles:
14        - values.yaml
15   destination:
16     server: https://kubernetes.default.svc
17     namespace: minio-operator
18   syncPolicy:
19     automated:
20       prune: true
21       selfHeal: true
22   syncOptions:
23     - CreateNamespace=true

```

Listing 11.3: *k8s/argocd/apps/minio-operator.yaml*

12 | Network Policies & Security Hardening

12.1 Default Deny All

```
1  ## Apply a default deny policy to EVERY platform namespace
2  ## This ensures only explicitly allowed traffic flows
3  ---
4  apiVersion: networking.k8s.io/v1
5  kind: NetworkPolicy
6  metadata:
7    name: default-deny-all
8    namespace: platform-ingestion
9  spec:
10   podSelector: {}      ## Applies to all pods in this namespace
11   policyTypes:
12     - Ingress
13     - Egress
14   ---
15   ## Repeat for every namespace (platform-storage, platform-compute, etc.)
16   apiVersion: networking.k8s.io/v1
17   kind: NetworkPolicy
18   metadata:
19     name: default-deny-all
20     namespace: platform-storage
21   spec:
22     podSelector: {}
23     policyTypes:
24       - Ingress
25       - Egress
```

Listing 12.1: *k8s/network-policies/default-deny.yaml*

```
1  ## Allow Spark (platform-compute) to reach MinIO (platform-storage)
2  apiVersion: networking.k8s.io/v1
3  kind: NetworkPolicy
4  metadata:
5    name: allow-from-compute
6    namespace: platform-storage
7  spec:
8    podSelector:
9      matchLabels:
10        app: minio      ## MinIO pods
11    policyTypes:
12      - Ingress
13    ingress:
14      - from:
```

```
15     - namespaceSelector:
16       matchLabels:
17         kubernetes.io/metadata.name: platform-compute
18     - namespaceSelector:
19       matchLabels:
20         kubernetes.io/metadata.name: platform-orchestr
21     - namespaceSelector:
22       matchLabels:
23         kubernetes.io/metadata.name: platform-serving
24   ports:
25     - protocol: TCP
26       port: 9000
```

Listing 12.2: *k8s/network-policies/allow-minio.yaml*

13 | Ansible Playbooks

```
1 ---
2 - name: Bootstrap Data Platform Post-Terraform
3   hosts: localhost
4   connection: local
5   gather_facts: false
6
7   vars:
8     cluster_name: "dataplatfom-dev"
9     aws_region:   "eu-west-1"
10
11   tasks:
12     - name: Update kubeconfig
13       command: >
14         aws eks update-kubeconfig
15           --region {{ aws_region }}
16           --name {{ cluster_name }}
17           --profile dataplatfom-dev
18
19     - name: Apply all Kubernetes namespaces
20       command: kubectl apply -f {{ playbook_dir }}/../k8s/namespaces/
21
22     - name: Apply RBAC manifests
23       command: kubectl apply -f {{ playbook_dir }}/../k8s/rbac/
24
25     - name: Apply StorageClasses
26       command: kubectl apply -f {{ playbook_dir }}/../k8s/storage-classes/
27
28     - name: Apply NetworkPolicies
29       command: kubectl apply -f {{ playbook_dir }}/../k8s/network-policies/
30
31     - name: Add Helm repositories
32       command: "{{ item }}"
33       loop:
34         - "helm repo add minio-operator https://operator.min.io"
35         - "helm repo add cnpg https://cloudnative-pg.github.io/charts"
36         - "helm repo add strimzi https://strimzi.io/charts/"
37         - "helm repo add spark-operator https://kubeflow.github.io/spark-
38           operator"
39         - "helm repo add apache-airflow https://airflow.apache.org"
40         - "helm repo add prometheus-community https://prometheus-community.
41           github.io/helm-charts"
42         - "helm repo add argo https://argoproj.github.io/argo-helm"
43         - "helm repo update"
```



```

42
43   - name: Install cert-manager
44     shell: |
45       helm upgrade --install cert-manager jetstack/cert-manager \
46         --namespace cert-manager --create-namespace \
47         --version v1.14.4 --set installCRDs=true
48     register: cert_manager_result
49     changed_when: "'already' not in cert_manager_result.stdout"
50
51   - name: Wait for cert-manager
52     command: >
53       kubectl rollout status deploy/cert-manager -n cert-manager --timeout
54         =120s
55
56   - name: Seed secrets into AWS Secrets Manager
57     aws_secret:
58       name: "datapatform/dev/{{ item.name }}"
59       state: present
60       region: "{{ aws_region }}"
61       secret: "{{ item.value }}"
62       loop: "{{ secrets_to_seed }}"    ## Passed via --extra-vars
63       no_log: true
64
65   - name: Verify all operators are healthy
66     hosts: localhost
67     connection: local
68     gather_facts: false
69     tasks:
70       - name: Check all platform pods are Running
71         shell: |
72           kubectl get pods --all-namespaces \
73             -l 'app.kubernetes.io/part-of=data-platform' \
74             --field-selector=status.phase!=Running 2>&1
75         register: unhealthy
76         failed_when: unhealthy.stdout | length > 0

```

Listing 13.1: *ansible/playbooks/bootstrap.yml*

```

1 cd ansible/
2
3 ## Run the full bootstrap (after Terraform apply)
4 ansible-playbook playbooks/bootstrap.yml \
5   --extra-vars "secrets_to_seed=[]" \
6   -v

```

Listing 13.2: *Run the bootstrap playbook*

14 | Validation & End-to-End Testing

14.1 Full Pipeline Smoke Test

```
1 #!/bin/bash
2 set -euo pipefail
3
4 echo "=== Data Platform E2E Validation ==="
5
6 ## 1. Kubernetes health
7 echo "[1/7] Checking all nodes are Ready..."
8 NOTREADY=$(kubectl get nodes --no-headers | grep -v " Ready" | wc -l)
9 [ "$NOTREADY" -eq 0 ] && echo " OK: All nodes Ready" || (echo " FAIL:
   $NOTREADY nodes not Ready" && exit 1)
10
11 ## 2. MinIO
12 echo "[2/7] Testing MinIO S3 API..."
13 kubectl run -it --rm minio-test --image=minio/mc:latest \
14   --restart=Never --namespace platform-storage -- \
15   mc alias set test http://minio:9000 minioadmin "${MINIO_ROOT_PASSWORD}" &&
   \
16   mc ls test/lakehouse && echo " OK: MinIO accessible, lakehouse bucket
   found"
17
18 ## 3. Hive Metastore
19 echo "[3/7] Checking Hive Metastore Thrift port..."
20 kubectl run -it --rm hms-test --image=busybox \
21   --restart=Never --namespace platform-metastore -- \
22   nc -zv hive-metastore 9083 && echo " OK: HMS Thrift port reachable"
23
24 ## 4. Kafka
25 echo "[4/7] Checking Kafka cluster..."
26 kubectl exec -n platform-ingestion \
27   platform-kafka-0 -- \
28   bin/kafka-topics.sh \
29     --bootstrap-server platform-kafka-bootstrap:9092 \
30     --list | grep bronze && echo " OK: Bronze topics exist"
31
32 ## 5. Submit a test Spark job
33 echo "[5/7] Submitting Spark Pi test job..."
34 cat <<'SPARKEOF' | kubectl apply -f -
35 apiVersion: sparkoperator.k8s.io/v1beta2
36 kind: SparkApplication
37 metadata:
38   name: spark-pi-test
39   namespace: platform-compute
```

```

40 spec:
41   type: Scala
42   mode: cluster
43   image: "apache/spark:3.5.1"
44   mainClass: org.apache.spark.examples.SparkPi
45   mainApplicationFile: "local:///opt/spark/examples/jars/spark-examples_2
46     .12-3.5.1.jar"
47   arguments: ["100"]
48   driver:
49     cores: 1
50     memory: "512m"
51     serviceAccount: spark-sa
52   executor:
53     cores: 1
54     instances: 1
55     memory: "512m"
56     restartPolicy:
57       type: Never
58 SPARKEOF
59 kubectl wait sparkapplication/spark-pi-test \
60   --for=jsonpath='{.status.applicationState.state}'=COMPLETED \
61   --timeout=300s -n platform-compute && echo "  OK: SparkPi completed"
62 kubectl delete sparkapplication/spark-pi-test -n platform-compute
63
64 ## 6. Airflow
65 echo "[6/7] Checking Airflow webserver..."
66 kubectl exec -n platform-orchestr \
67   deploy/airflow-webserver -- \
68   airflow version && echo "  OK: Airflow webserver responding"
69
70 ## 7. Prometheus metrics
71 echo "[7/7] Checking Prometheus targets..."
72 kubectl exec -n platform-monitoring \
73   $(kubectl get pod -n platform-monitoring -l app=prometheus -o jsonpath='{.
74     items[0].metadata.name}') -- \
75   wget -qO- "http://localhost:9090/api/v1/targets" | \
76   python3 -c "import sys,json; d=json.load(sys.stdin); \
77   up=[t for t in d['data']['activeTargets'] if t['health']=='up']; \
78   print(f'  OK: {len(up)} Prometheus targets are UP')"
79
80 echo ""
81 echo "=== ALL CHECKS PASSED ==="

```

Listing 14.1: `scripts/e2e_test.sh` – End-to-end validation

14.2 Iceberg Table Verification

```

1 ## Start an interactive Spark shell pod
2 kubectl run -it --rm spark-shell \
3   --image=123456789.dkr.ecr.eu-west-1.amazonaws.com/spark-iceberg:3.5.1 \
4   --namespace platform-compute \
5   --env="SPARK_EXTRA_CLASSPATH=/opt/spark/jars/*" \

```

```

6  -- \
7  /opt/spark/bin/spark-shell \
8      --conf spark.sql.extensions=org.apache.iceberg.spark.extensions.
          IcebergSparkSessionExtensions \
9      --conf spark.sql.catalog.lakehouse=org.apache.iceberg.spark.SparkCatalog
          \
10     --conf spark.sql.catalog.lakehouse.type=hive \
11     --conf spark.sql.catalog.lakehouse.uri=thrift://hive-metastore.platform-
          metastore:9083 \
12     --conf spark.hadoop.fs.s3a.endpoint=http://minio.platform-storage:9000 \
13     --conf spark.hadoop.fs.s3a.path.style.access=true
14
15 ## In the Spark shell:
16 ## spark.sql("SHOW NAMESPACES IN lakehouse").show()
17 ## spark.sql("SHOW TABLES IN lakehouse.bronze").show()
18 ## spark.sql("SELECT COUNT(*) FROM lakehouse.bronze.agency_budget").show()
19 ## spark.sql("SELECT * FROM lakehouse.bronze.agency_budget.snapshots").show
    ()

```

Listing 14.2: *Verify Iceberg tables via Spark shell***✓ Final Checkpoint: Platform Fully Operational**

- ✓ All 9 platform namespaces active with network isolation
- ✓ MinIO 8TiB Lakehouse with Bronze/Silver/Gold Iceberg tables
- ✓ Kafka 3-broker cluster; CDC flowing from PostgreSQL
- ✓ Spark Operator; dbt Silver & Gold transforms running
- ✓ Airflow orchestrating daily pipelines via Git-synced DAGs
- ✓ Dremio federating Gold tables to Power BI
- ✓ Cloudflare Zero Trust exposing 5 services (no open ports)
- ✓ Prometheus + Grafana; Fluent Bit + OpenObserve operational
- ✓ GitHub Actions CI/CD: lint → plan → apply → smoke test
- ✓ Lambda scaling EKS nodes 08:00–18:00 (save ≈60% compute cost)
- ✓ ArgoCD reconciling cluster state to Git (optional)

Quick Reference Cheat Sheet

Task	Command
Get all pod statuses	<code>kubect1 get pods -A grep -v Running</code>
Watch Spark job	<code>kubect1 get sparkapplication -n platform-compute -w</code>
Check MinIO health	<code>mc admin info lakehouse</code>
Kafka topic list	<code>kubect1 exec -n platform-ingestion platform-kafka-0 - bin/kafka-topics --zookeeper-quorum-scheme=s3 --zookeeper-quorum-servers=s3://lakehouse-1.s3.amazonaws.com:9000</code>
Airflow trigger DAG	<code>kubect1 exec -n platform-orchestr deploy/airflow-webserver - airflow trigger dag_id</code>
dbt run Silver	<code>dbt run -models silver -target dev</code>
dbt run Gold	<code>dbt run -models gold -target dev</code>
Check Prometheus targets	<code>kubect1 port-forward svc/monitoring-prometheus -n platform-monitoring 9090</code>
Restart HMS	<code>kubect1 rollout restart deploy/hive-metastore -n platform-metastore</code>
Scale compute nodes up	<code>aws eks update-nodegroup-config --cluster-name dataplatforn-dev --nodegroup-name dataplatforn-dev-nodes --desired-size 10</code>
Force scale down	<code>aws eks update-nodegroup-config --cluster-name dataplatforn-dev --nodegroup-name dataplatforn-dev-nodes --desired-size 0</code>
Check CF tunnel	<code>kubect1 get clustertunnel -n platform-security</code>
ArgoCD sync all	<code>argocd app sync data-platform-root -cascade</code>
Destroy everything	<code>terraform destroy -auto-approve</code> (from environments/dev/)